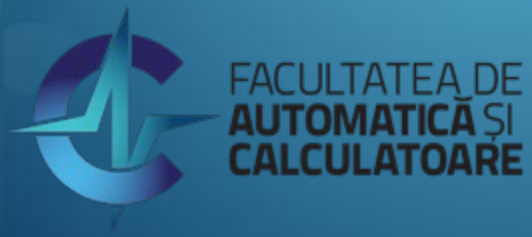


Algoritmi Paraleli și Distribuiți
Complexitate algoritmi paraleli

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro





Performanța

- Timp de execuție
- Memorie ocupată
- Număr de procese (thread-uri)
- Scalabilitate
- Toleranță la defecte
- Cost
- Eficiență
- Fiabilitate
- Portabilitate

Măsuri

- T - Timpul total necesar execuției programului paralel
- P - Numărul de procesoare utilizate
- G – Timp execuție **cel mai rapid algoritm secvențial**
- S – Speedup
  - $S = \frac{G}{T}$

- Costul C = T * P

- Eficienţa $E = \frac{G}{C} = \frac{G}{TP} = \frac{S}{P}$

Complexitate



* 3



Complexitate secvențială?

Complexitate



* 3

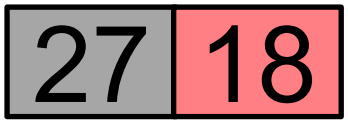
27

Complexitate secvențială?

Complexitate



* 3



Complexitate secvențială?

Complexitate



* 3

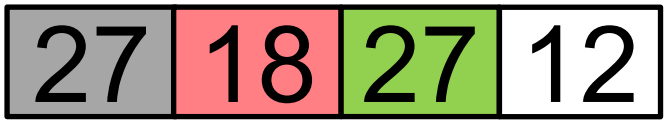


Complexitate secvențială?

Complexitate



* 3



Complexitate secvențială?

Complexitate



* 3



Complexitate secvențială? O(**N**)

Complexitate



* 3

Complexitate paralelă?

Complexitate



* 3



Complexitate paralelă? **O(1)** ?

Complexitate



* 3



Complexitate paralelă? **O(1)** ? **P = N**

Complexitate



* 3



Complexitate paralelă? $\mathbf{O}(\frac{N}{P})$

Complexitate



Speedup?

Complexitate



Speedup?

T = O($\frac{N}{P}$)

Complexitate



Speedup?

T = O($\frac{N}{P}$)

G = O(N)

Complexitate



Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})}$$

Complexitate



Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Complexitate



$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Eficiența?

T = O($\frac{N}{P}$)

G = O(N)

Complexitate



$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Eficiența?

$$E = \frac{S}{P} = \frac{O(P)}{P} = 1$$

T = O($\frac{N}{P}$)

G = O(N)

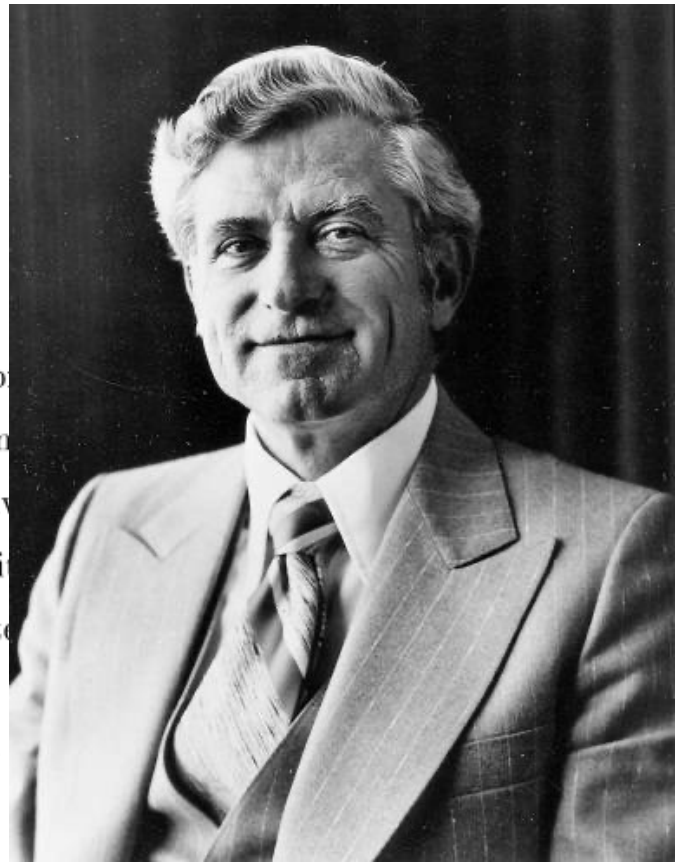
Legea lui Amdhal

Amdahl's law

Validity of the single processor approach to achieving large scale computing capabilities[1]

Gene M. Amdahl

IBM Sunnyvale, California



1 INTRODUCTION

For over a decade prophets have voiced the contention that the o
has reached its limits and that truly significant advences can be m
multiplicity of computers in such a manner as to permit cooperati
direction has been pointed out as general purpose computers wi
of memories, or as specialized computers with geometrically relat
controlled by one or more instruction streams.

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

$$S = \frac{G}{T}$$

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

$$S = \frac{G}{T} = \frac{G}{fG + \frac{(1-f)G}{P}} = \frac{1}{f + \frac{(1-f)}{P}}$$

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{S \leq \frac{1}{f}}$$

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{T \geq fG}$$

Legea lui Amdahl

Porțiune Secvențială

Porțiune Paralelizabilă

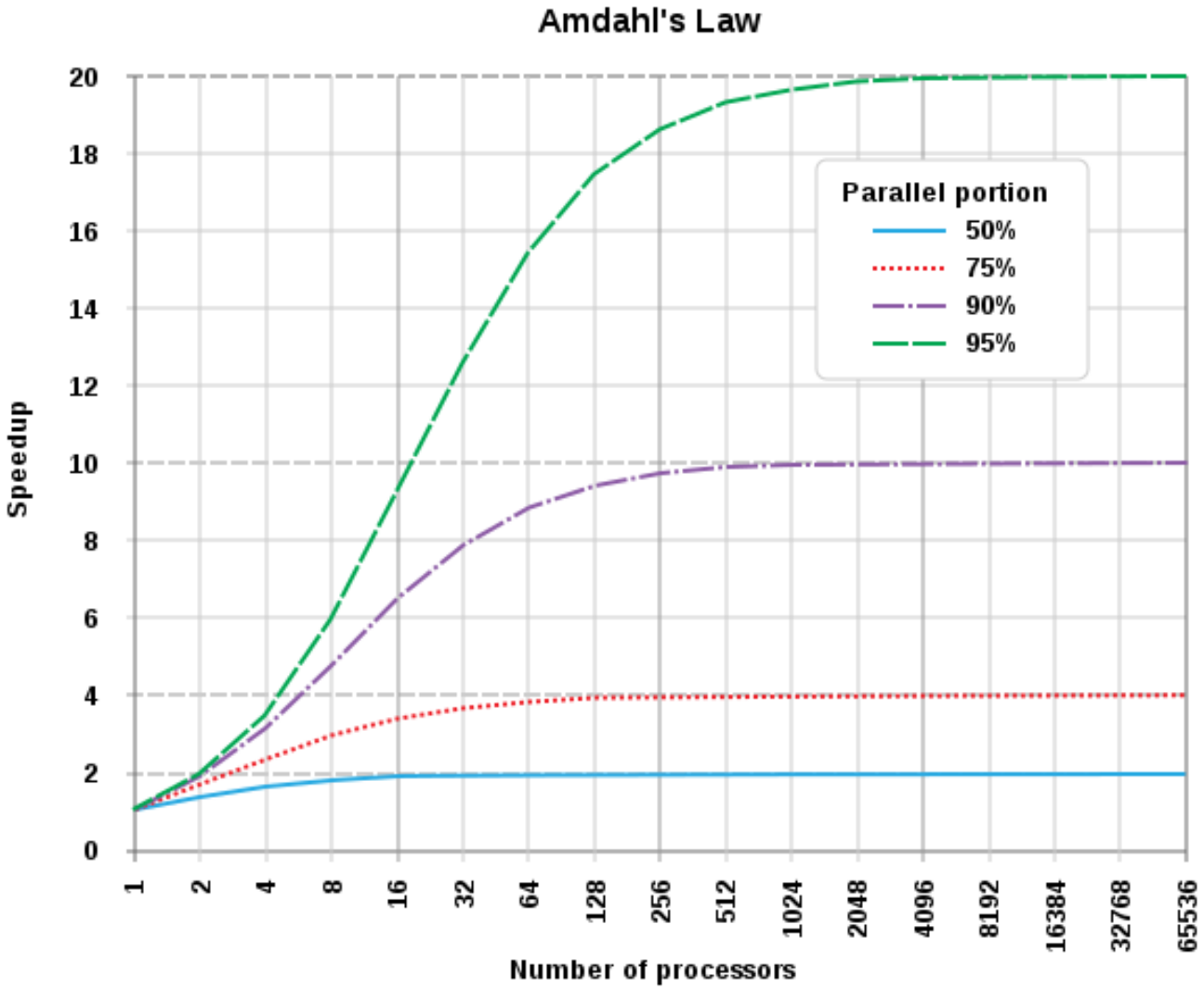


Porțiune Secvențială

Porțiune Paralelizată

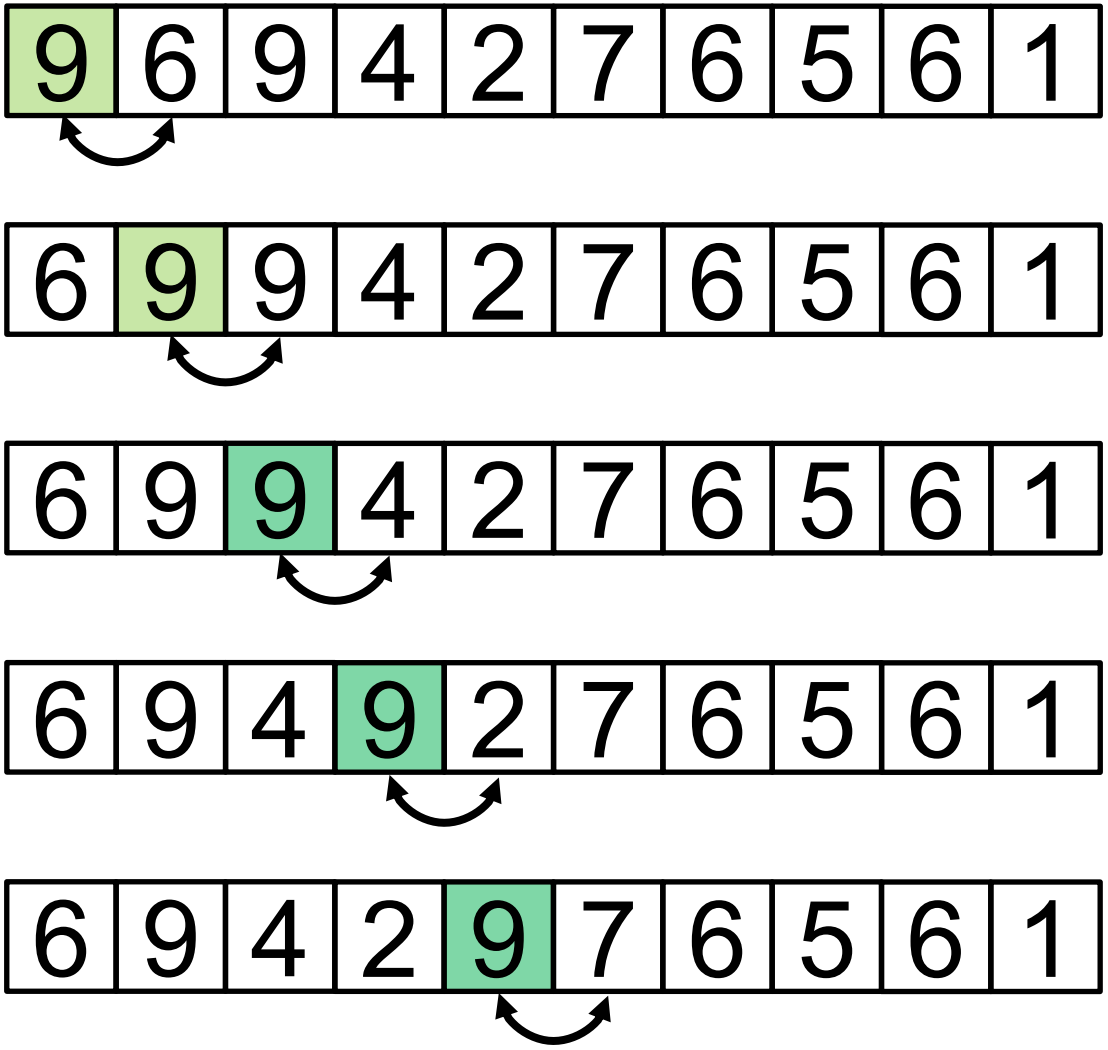


Legea lui Amdahl

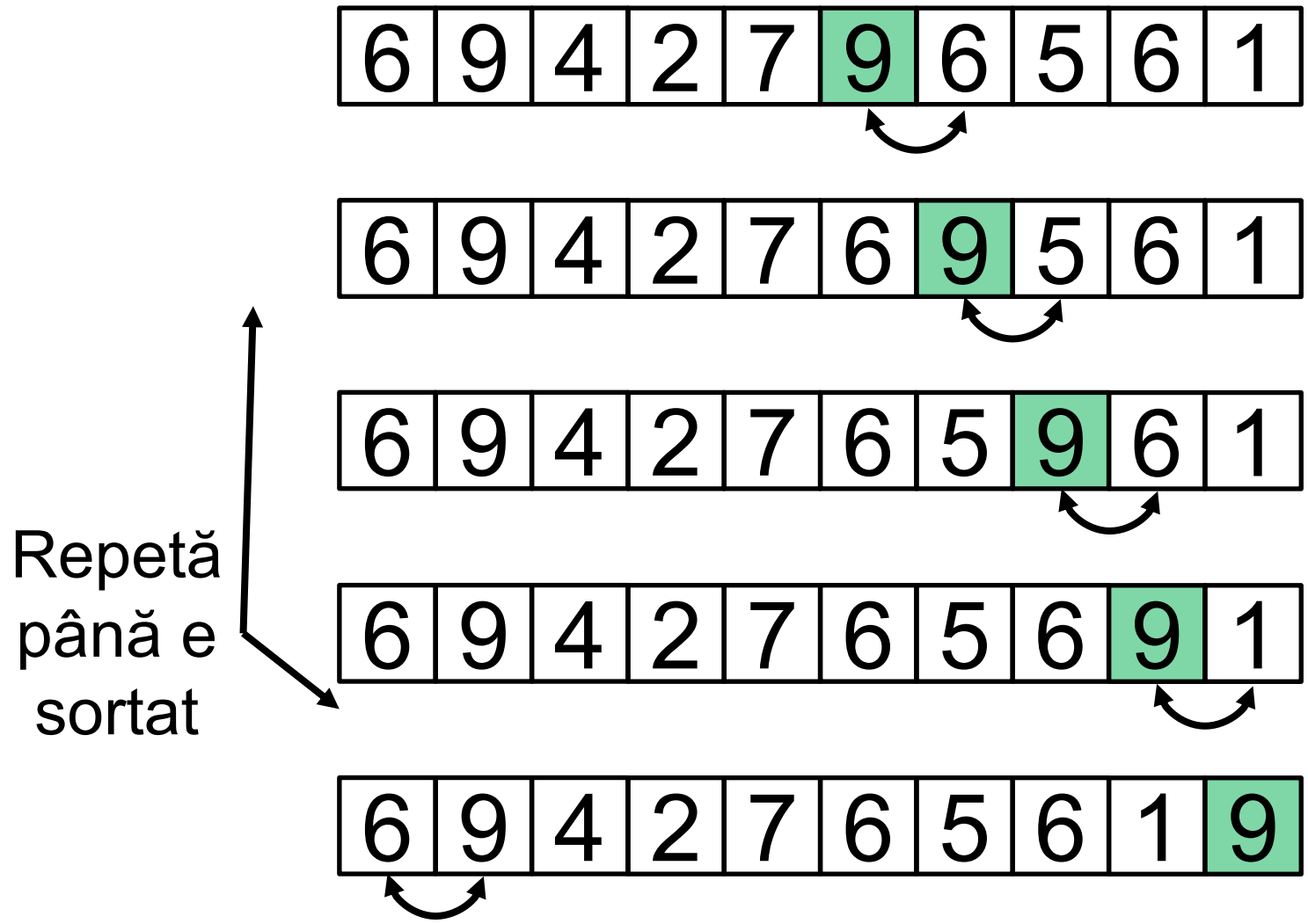


Exemplu: sortarea unui vector

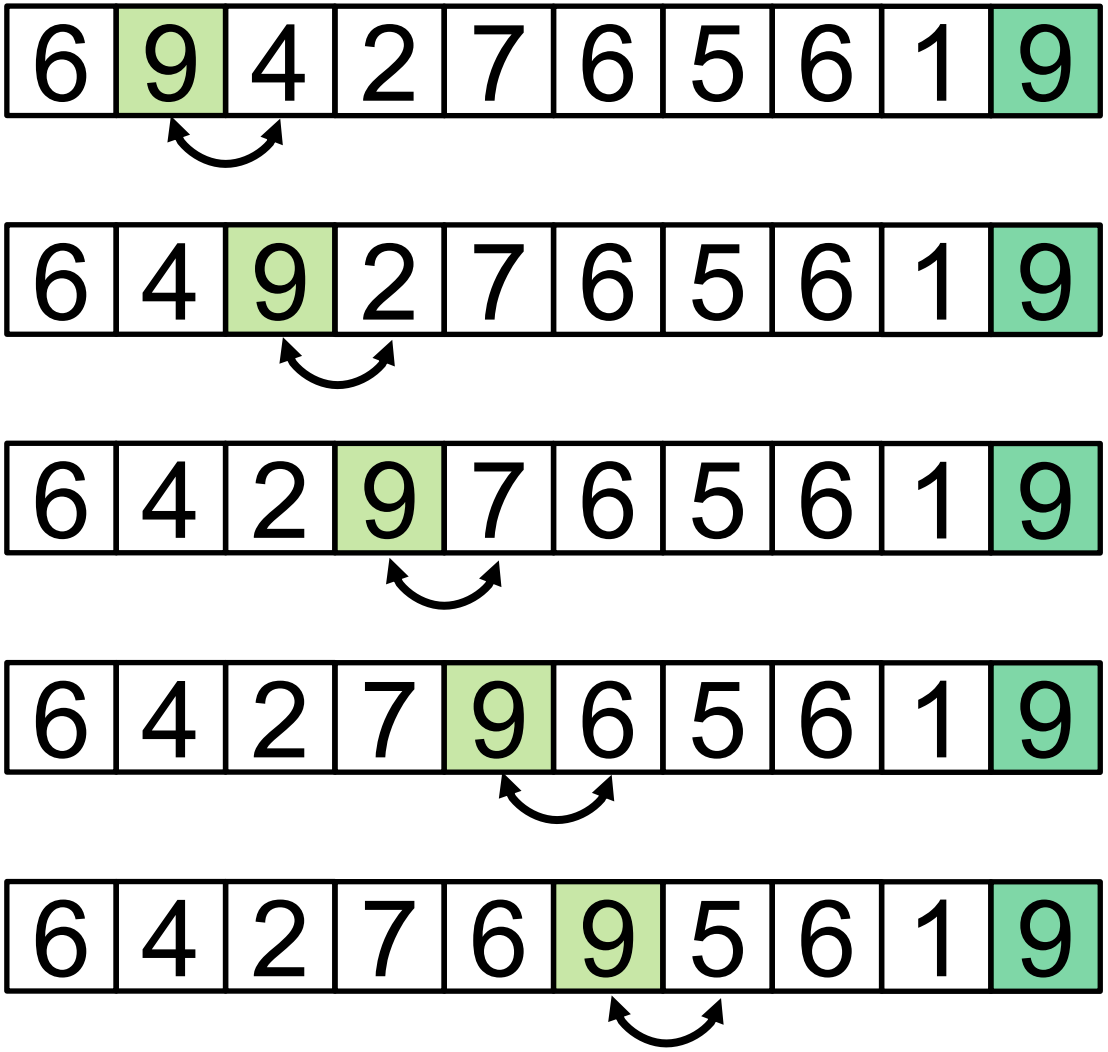
Bubble sort


9 6 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 4 9 2 7 6 5 6 1
6 9 4 2 9 7 6 5 6 1

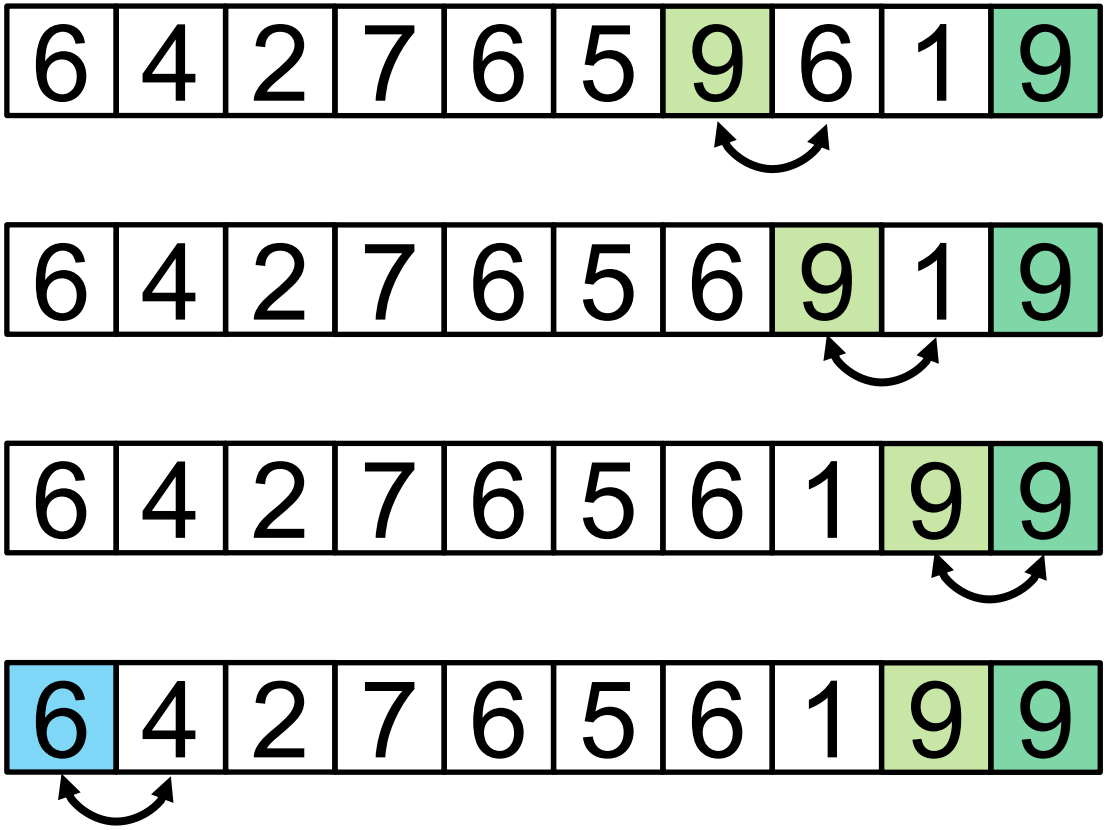
Bubble sort


6 9 4 2 7 9 6 5 6 1
6 9 4 2 7 6 9 5 6 1
6 9 4 2 7 6 5 9 6 1
6 9 4 2 7 6 5 6 9 1
6 9 4 2 7 6 5 6 1 9
Repetă până e sortat

Bubble sort

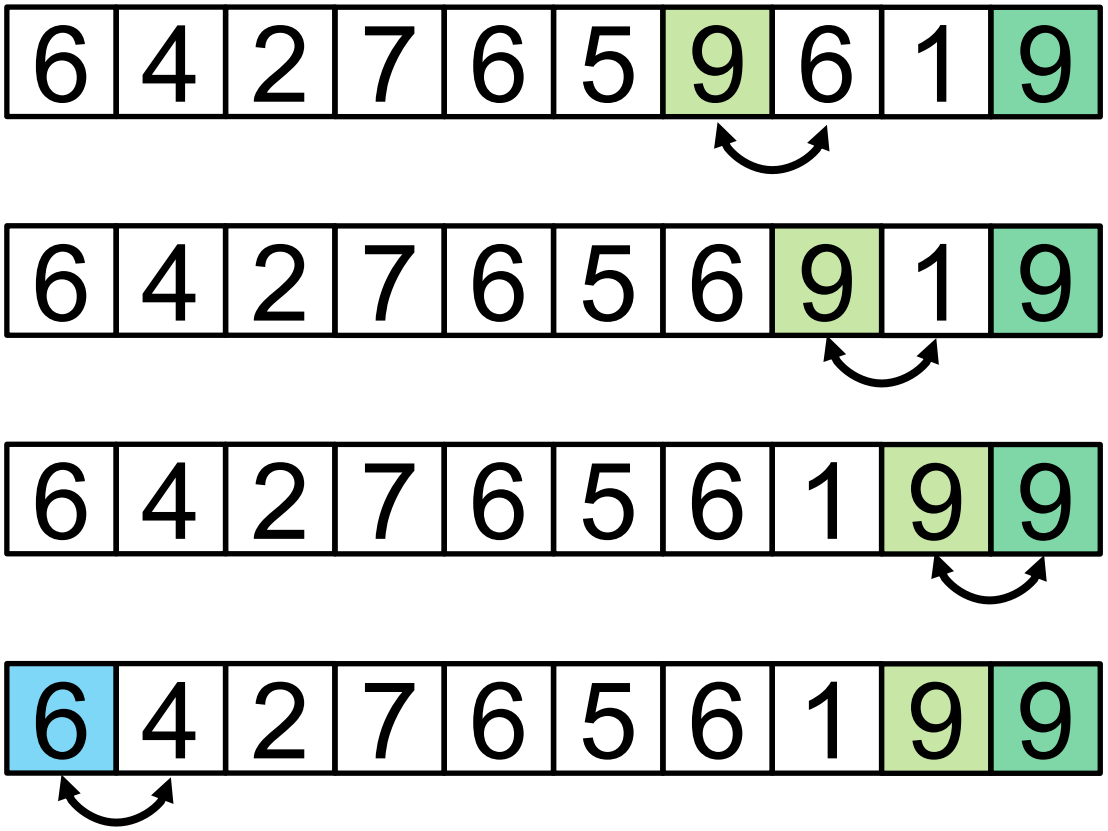

6 9 4 2 7 6 5 6 1 9
6 4 9 2 7 6 5 6 1 9
6 4 2 9 7 6 5 6 1 9
6 4 2 7 9 6 5 6 1 9
6 4 2 7 6 9 5 6 1 9

Bubble sort



.....

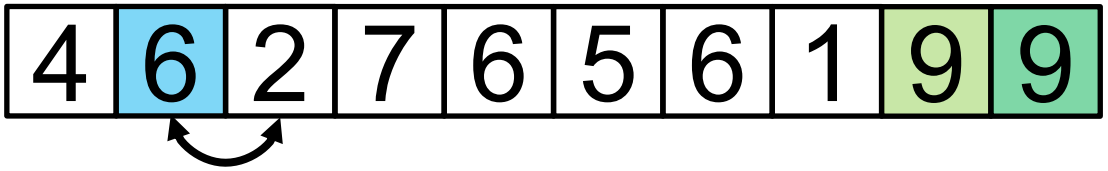
Bubble sort



.....

Complexitate?

Bubble sort

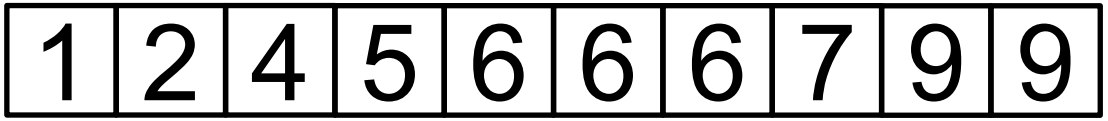


.....

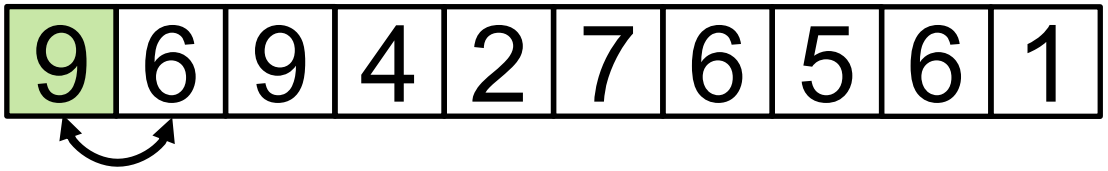
Complexitate: **O(n^2)**

Un pas trece prin toate elementele

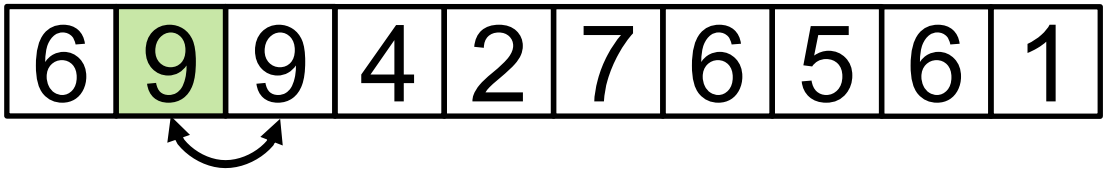
Garantat să termine după n repetiții



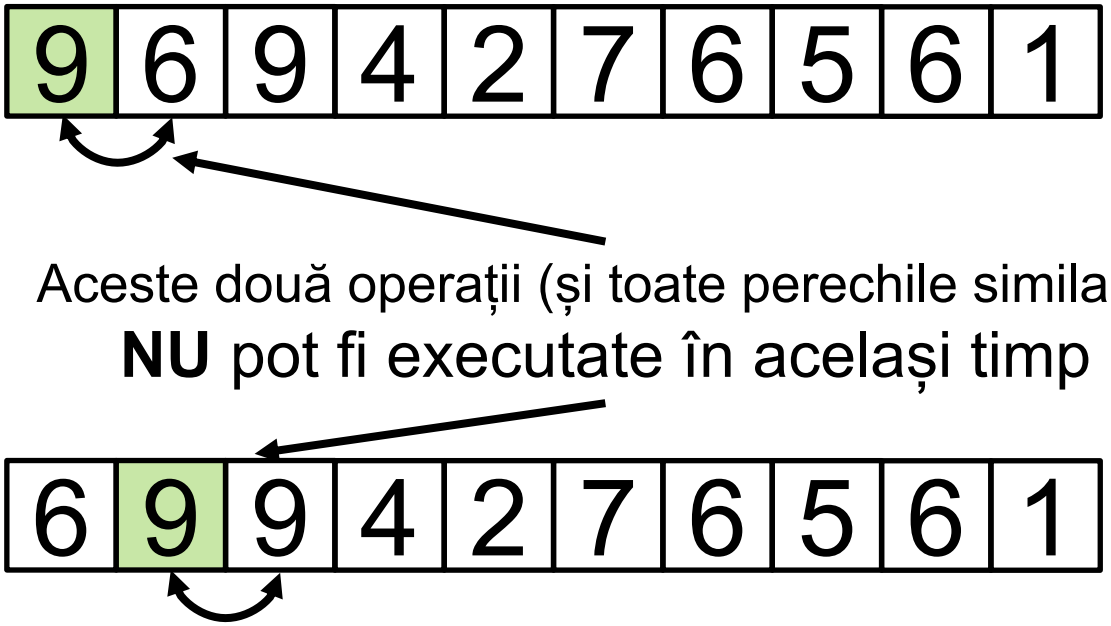
Parallel bubble sort



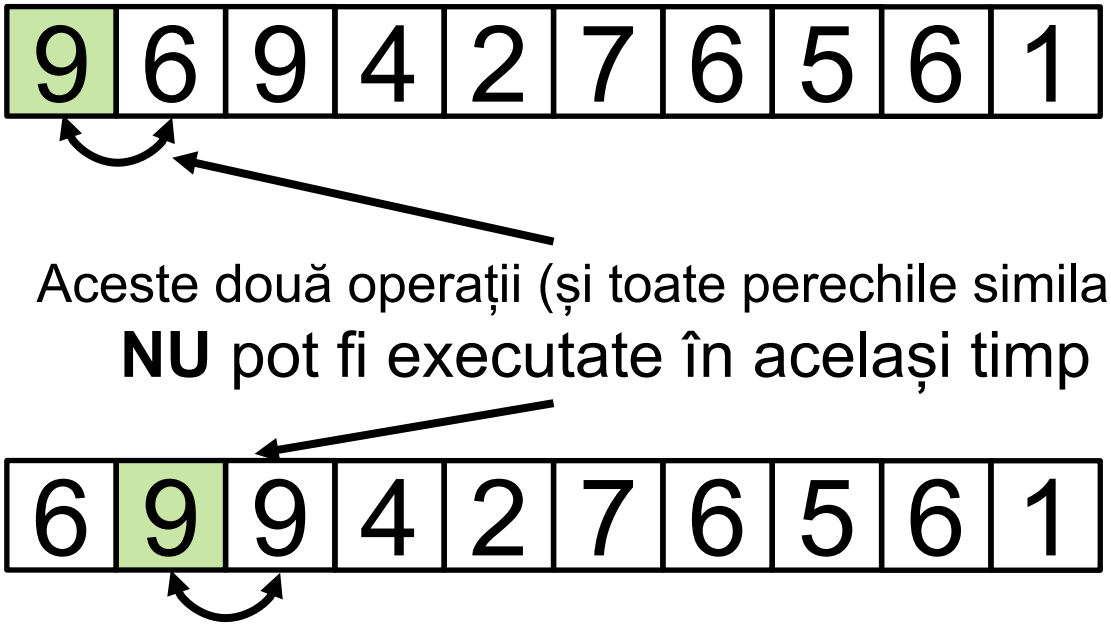
Cum paralelizăm?



Parallel bubble sort



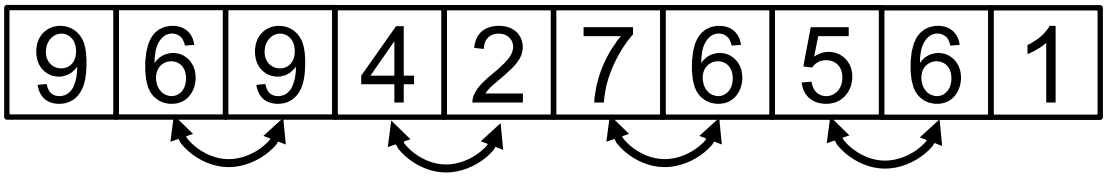
Parallel bubble sort



Solution hint: Nu e necesară execuția operațiilor într-o anumită ordine

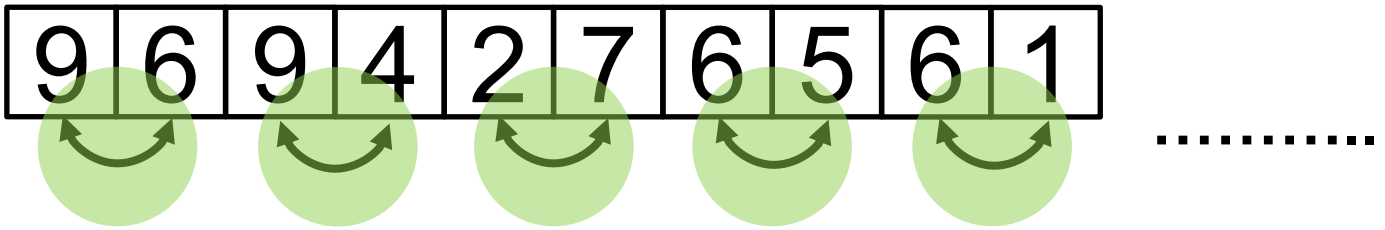
OETS: Odd-Even Transposition Sort



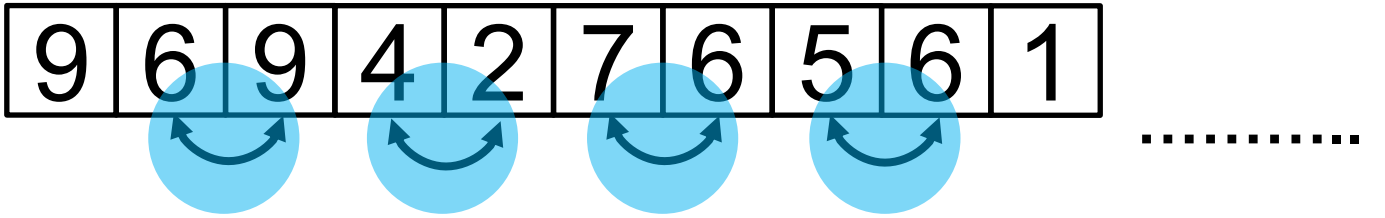


OETS: Odd-Even Transposition Sort

Pot fi executate în paralel →



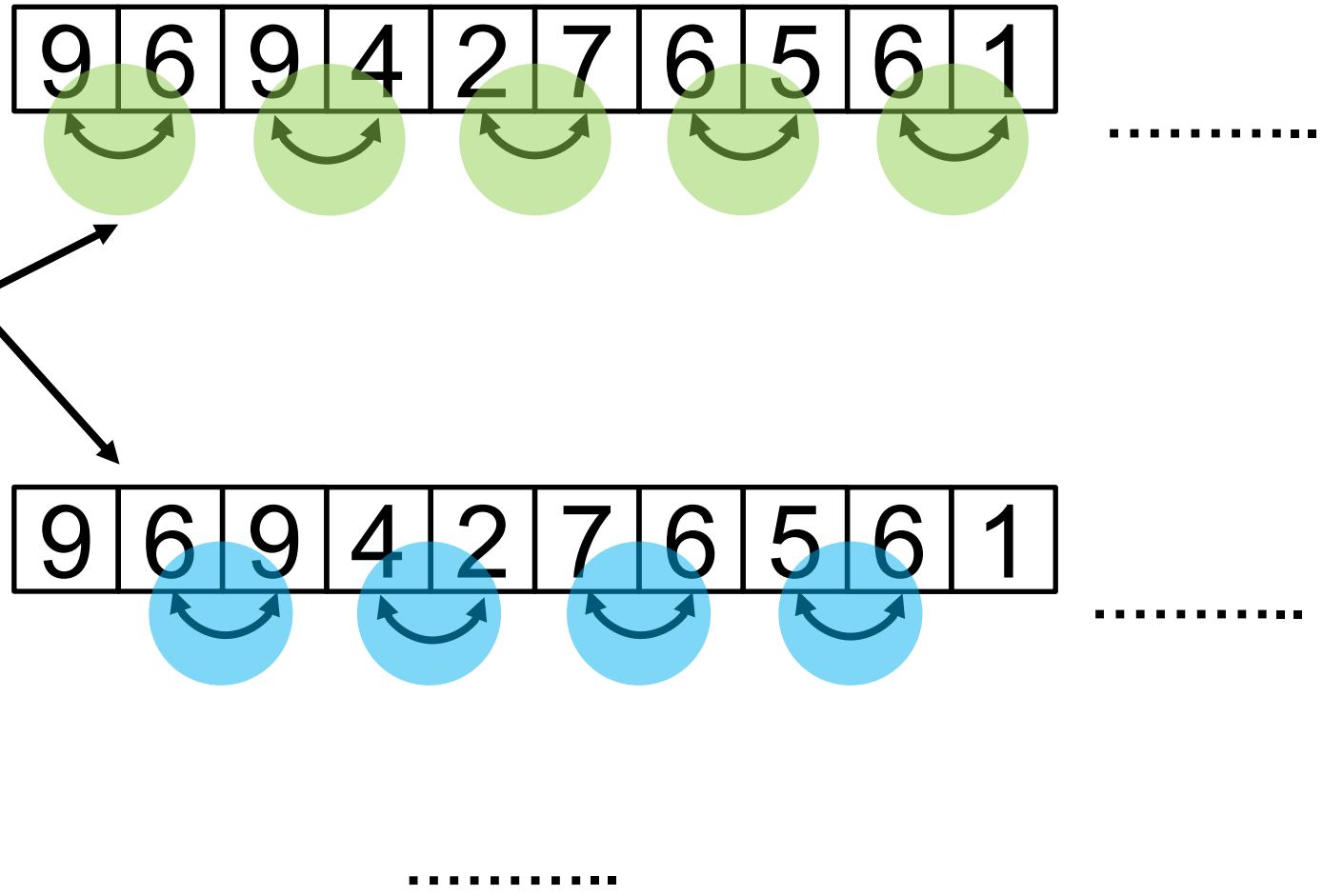
Pot fi executate în paralel →



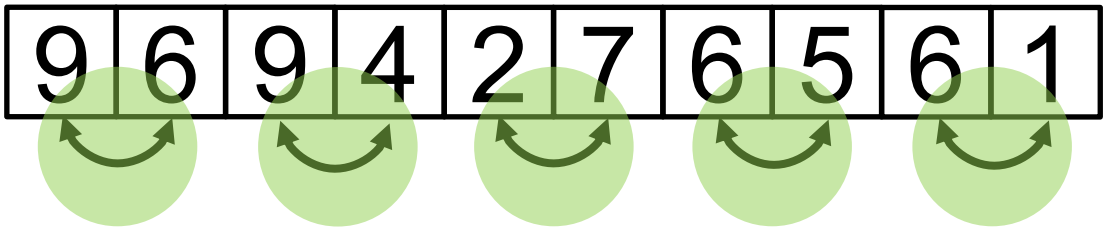
.....

.....

OETS: Odd-Even Transposition Sort

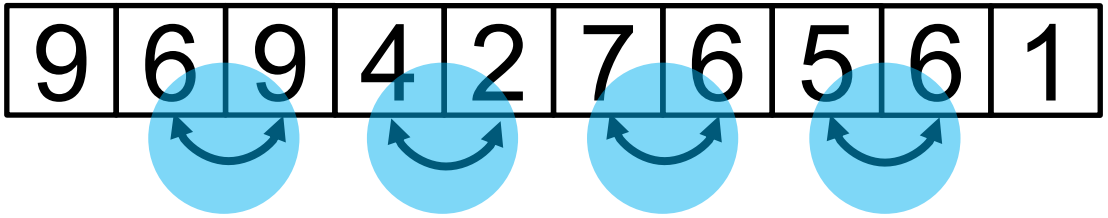


OETS: Odd-Even Transposition Sort



.....

NU pot fi executate în paralel

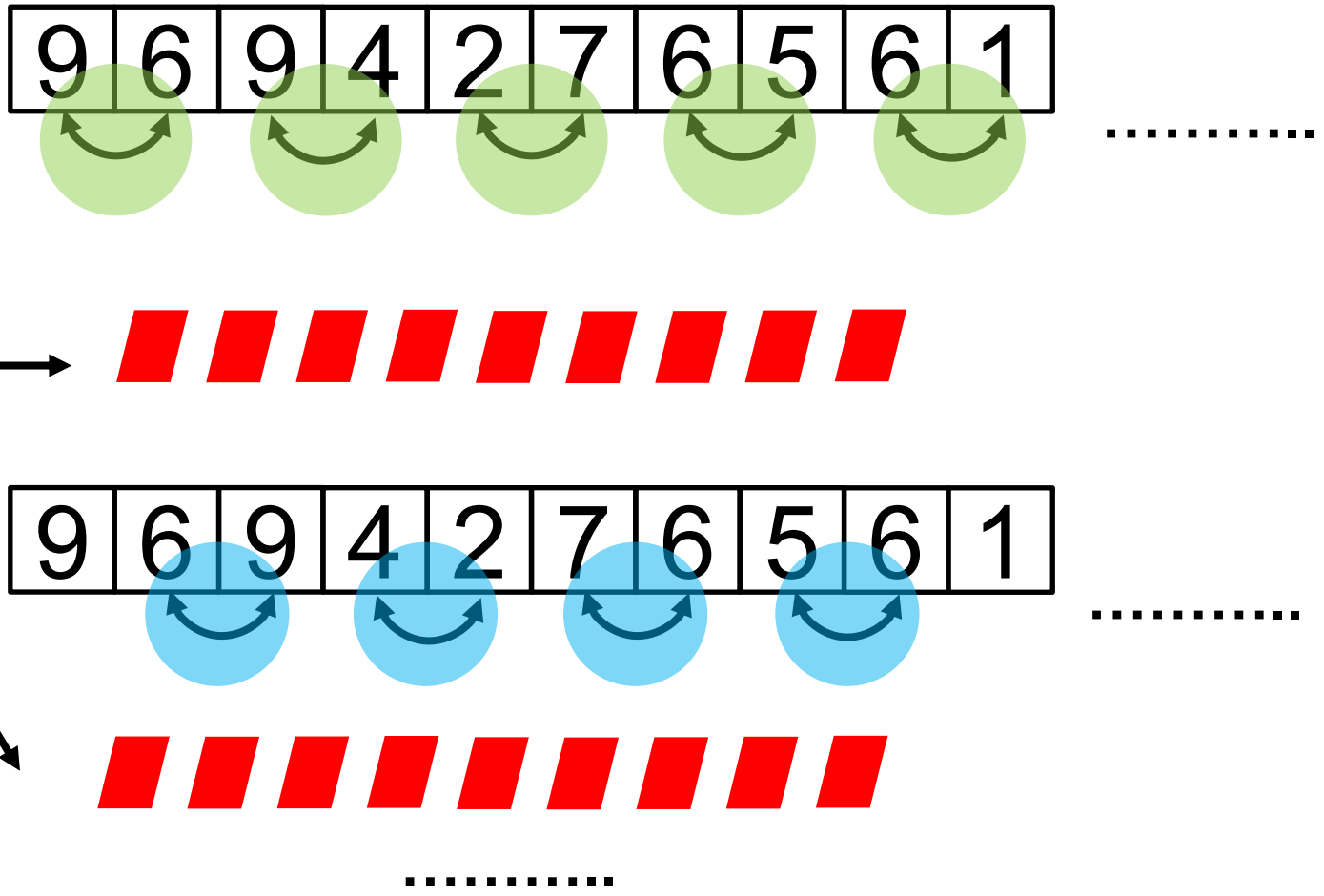


.....

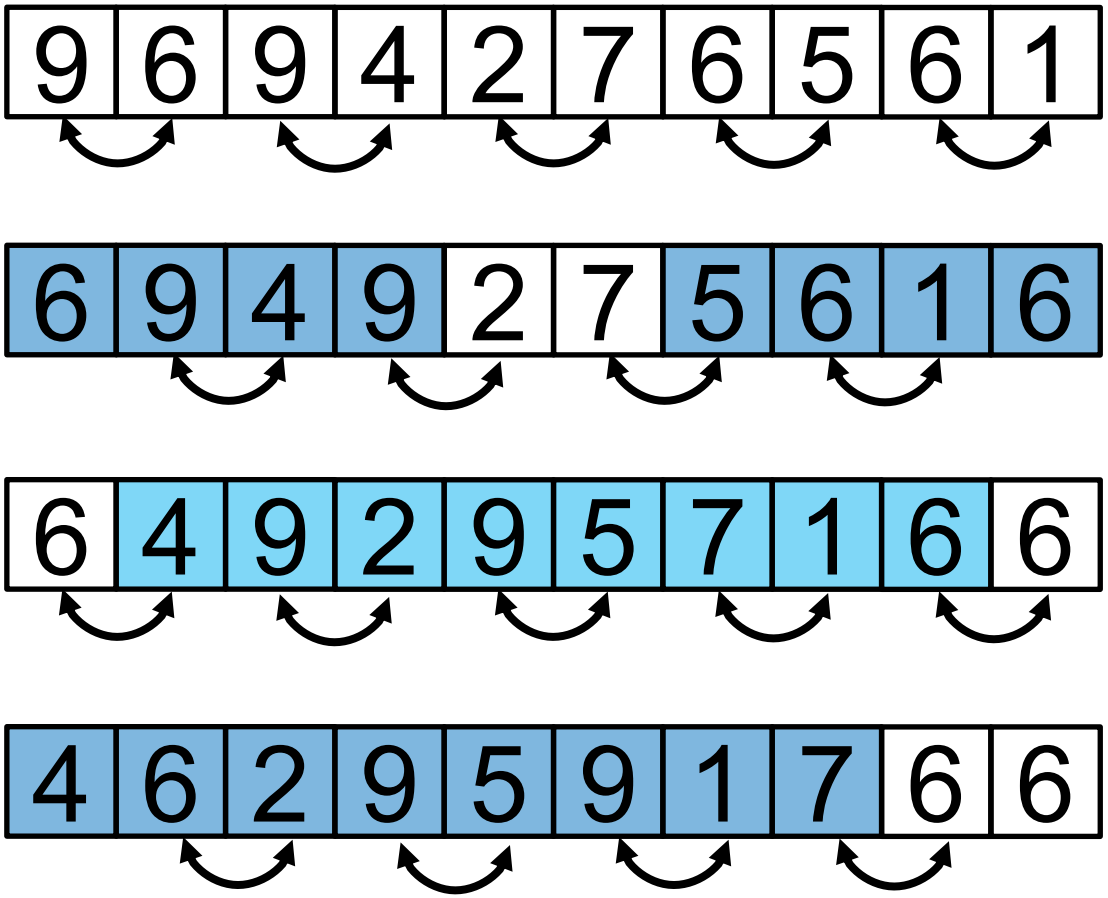
Ce facem? Ce folosim?

.....

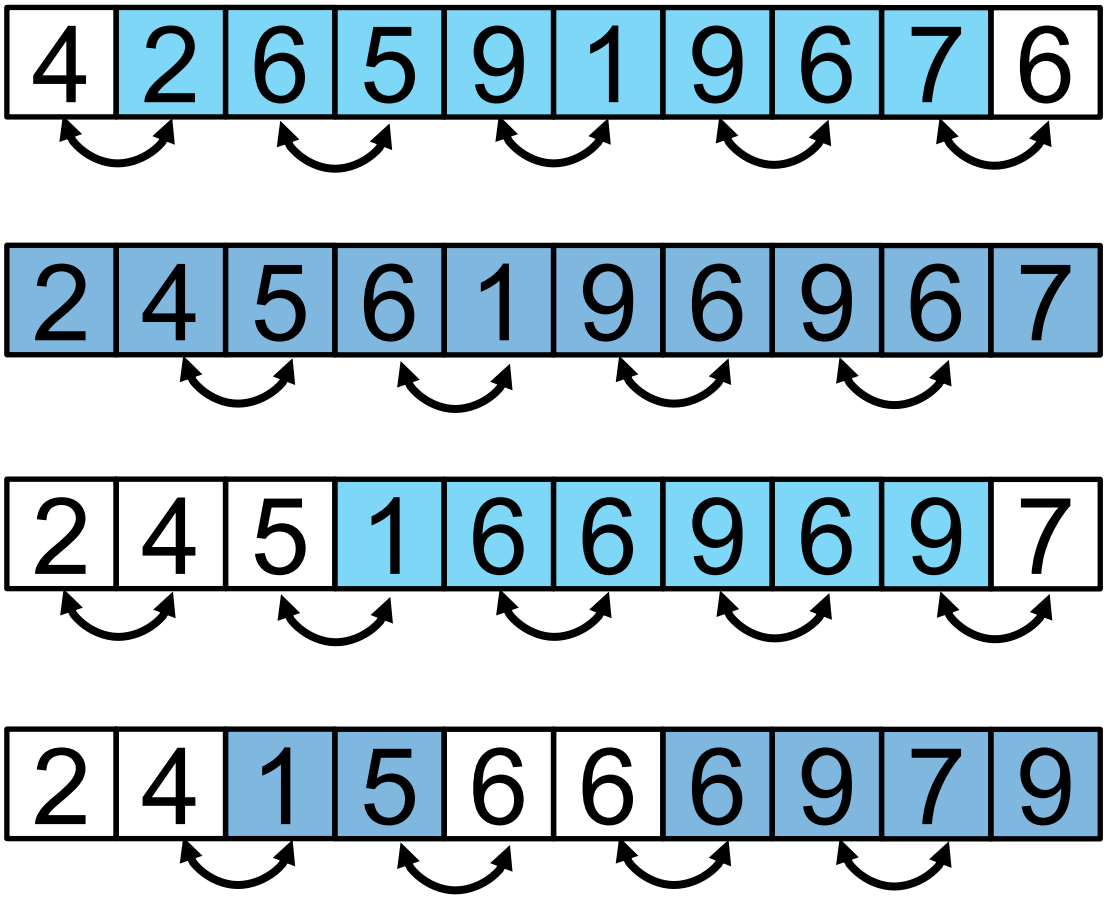
OETS: Odd-Even Transposition Sort



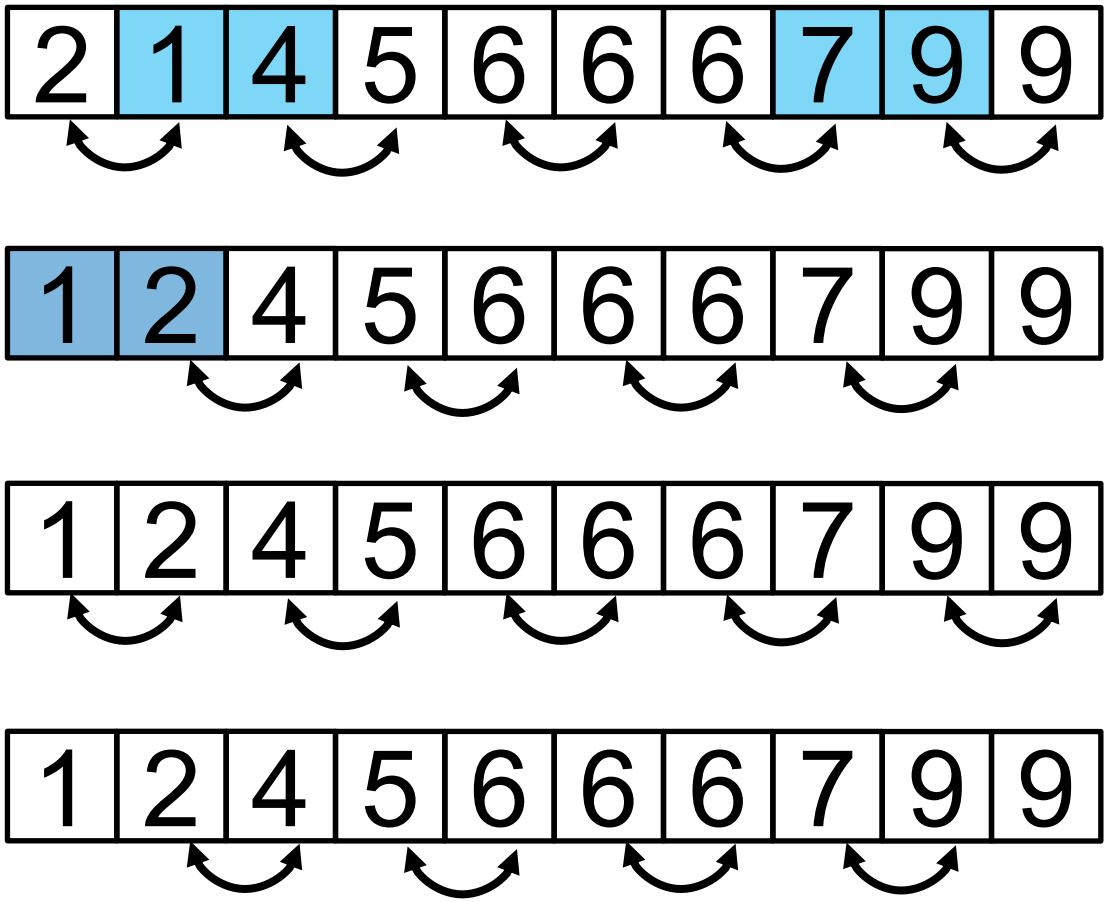
OETS: Odd-Even Transposition Sort



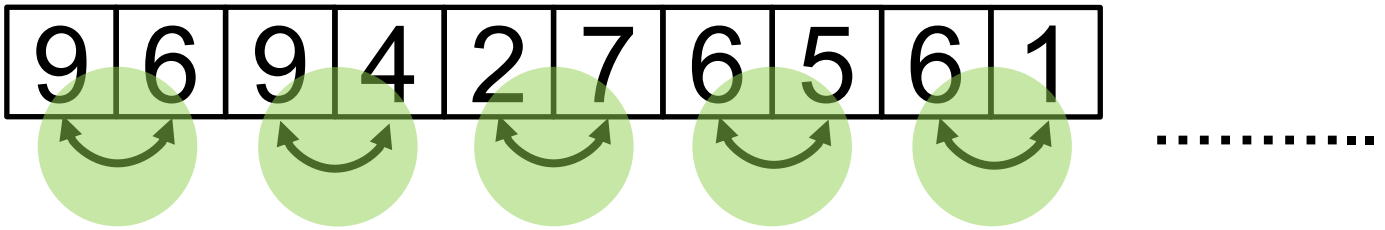
OETS: Odd-Even Transposition Sort



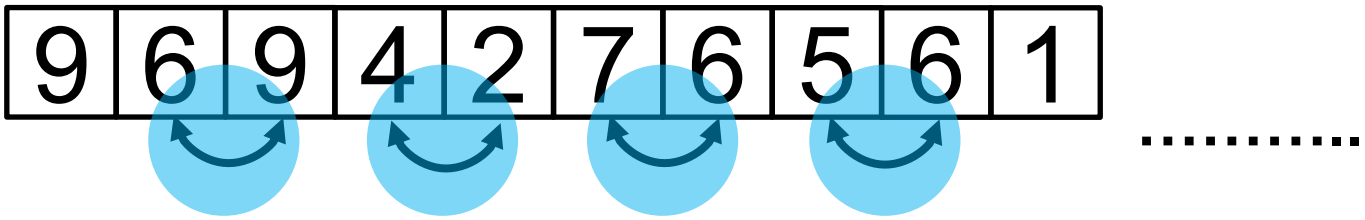
OETS: Odd-Even Transposition Sort



OETS: Odd-Even Transposition Sort

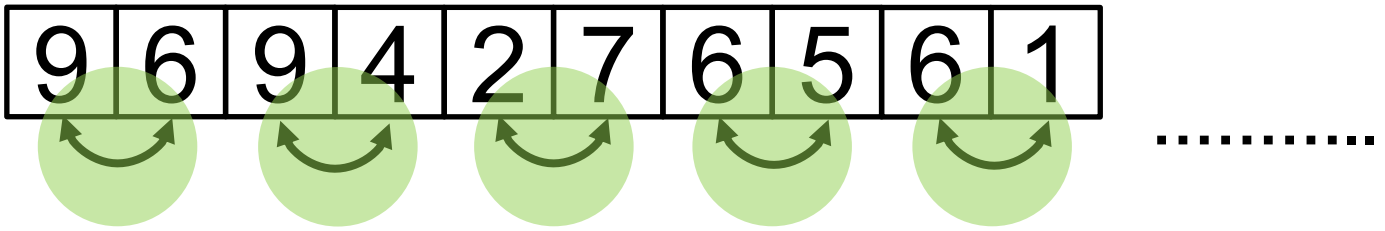


Complexitate a soluției paralele?

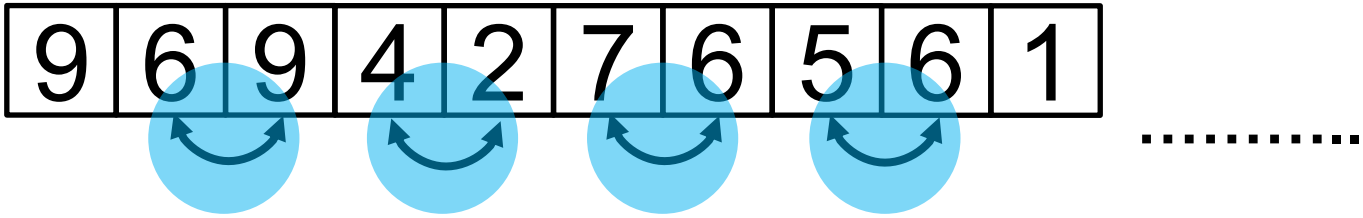


.....

OETS: Odd-Even Transposition Sort

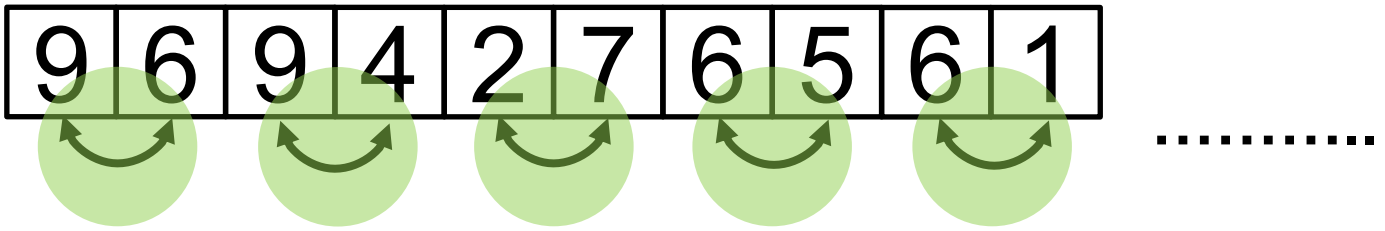


Complexitate a soluției paralele?

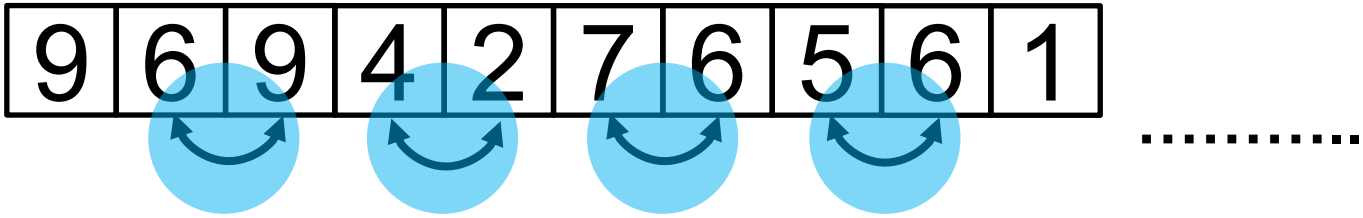


$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

OETS: Odd-Even Transposition Sort



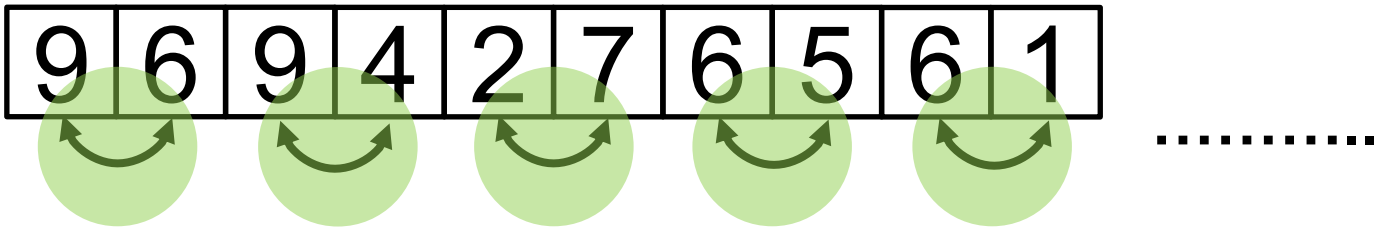
Speedup?



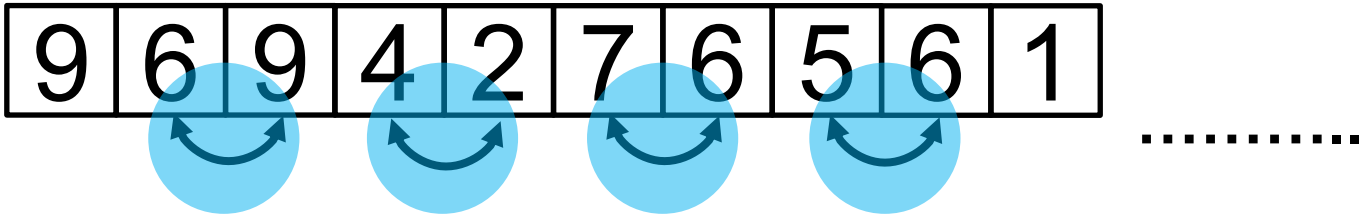
$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

.....

OETS: Odd-Even Transposition Sort



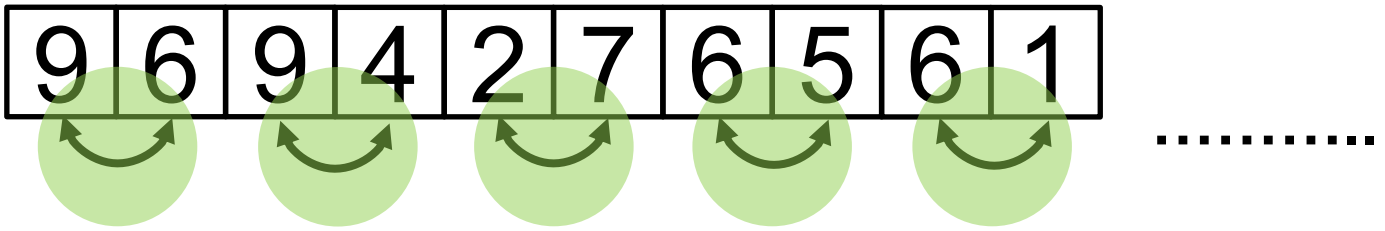
Speedup?



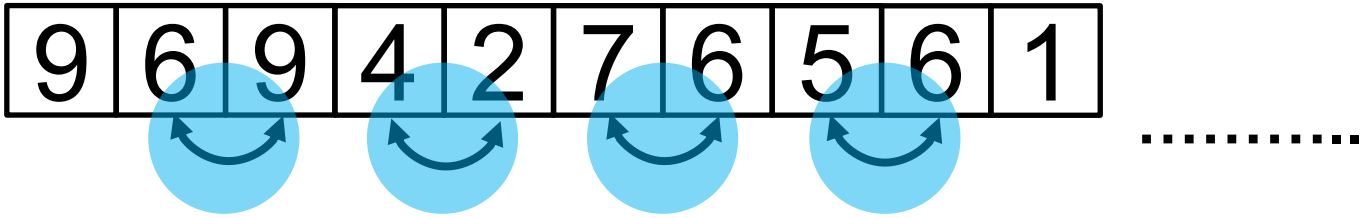
$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

.....

OETS: Odd-Even Transposition Sort

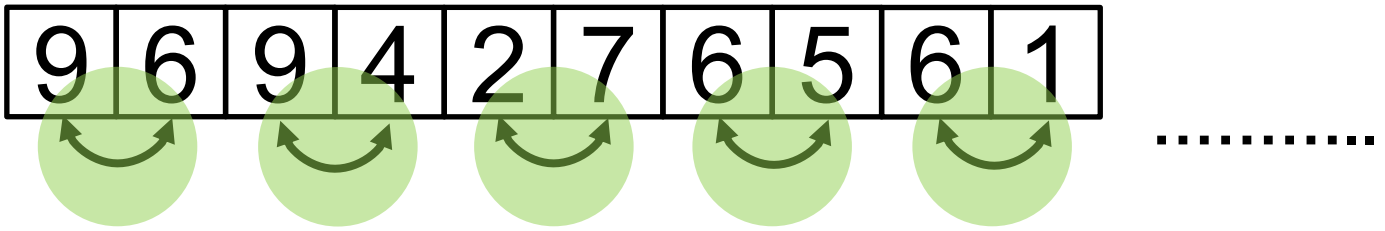


Speedup? **Sigur**?



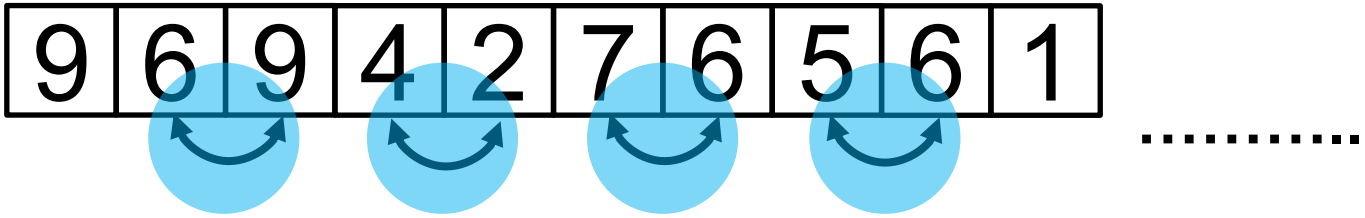
$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

OETS: Odd-Even Transposition Sort



.....

Speedup?

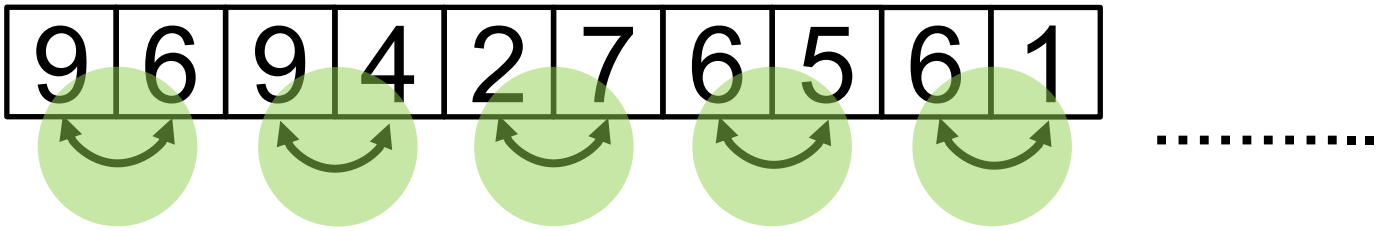


.....

$$S = \frac{N log_2 N}{\frac{N^2}{P}}$$

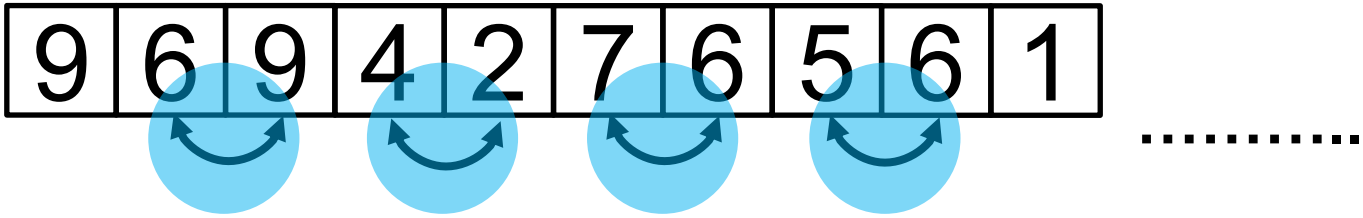
.....

OETS: Odd-Even Transposition Sort



.....

Speedup?

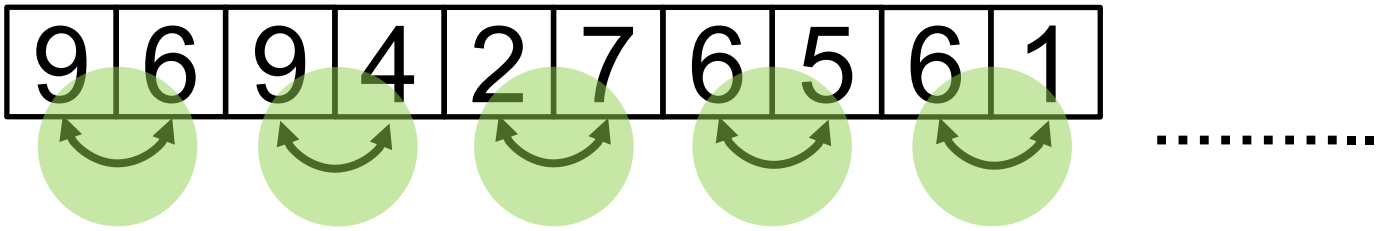


.....

$$S = \frac{Plog_2 N}{N}$$

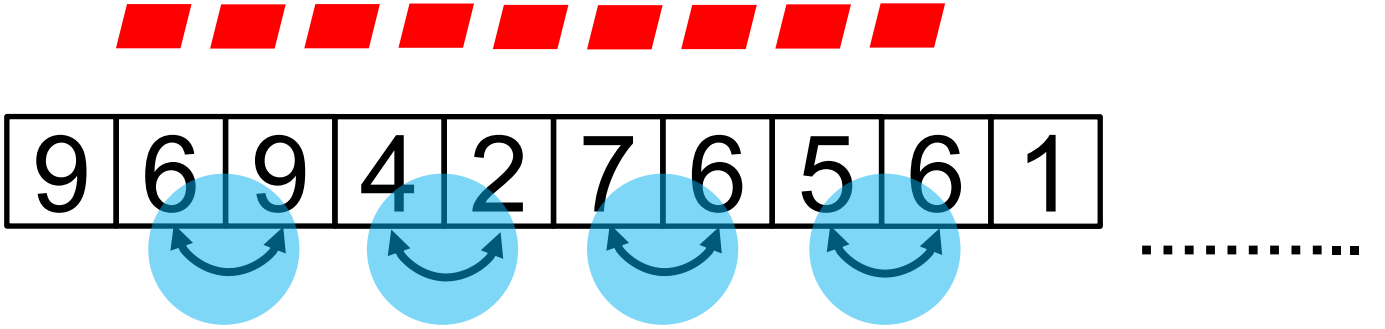
.....

OETS: Odd-Even Transposition Sort



.....

Nu uitați: Așteptatul la barieră poate introduce timpi mari!

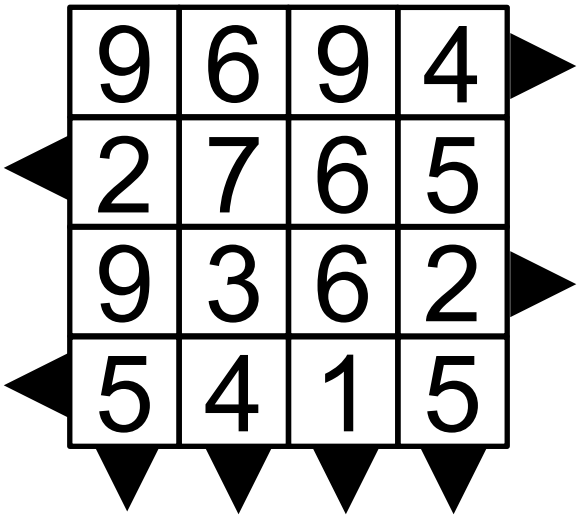


.....

$$S = \frac{Plog_2N}{N}$$

.....

Shear sort
(Row-column sort)
(Snake sort)



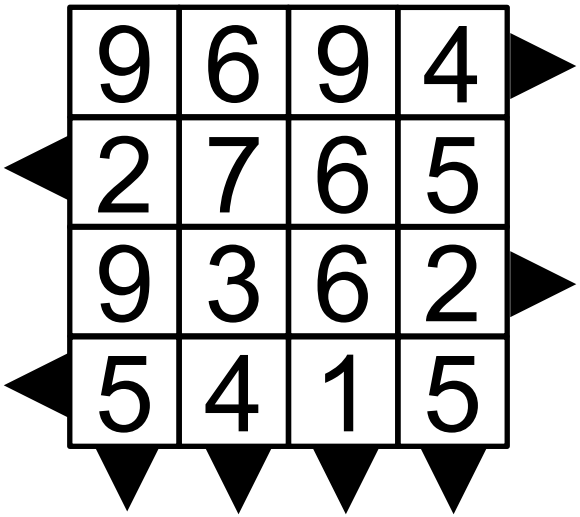
Sortează fiecare linie **pară** în mod **ascendent**
Sortează fiecare linie **impară** în mod **descendent**

Shear sort

9	6	9	4
2	7	6	5
9	3	6	2
5	4	1	5

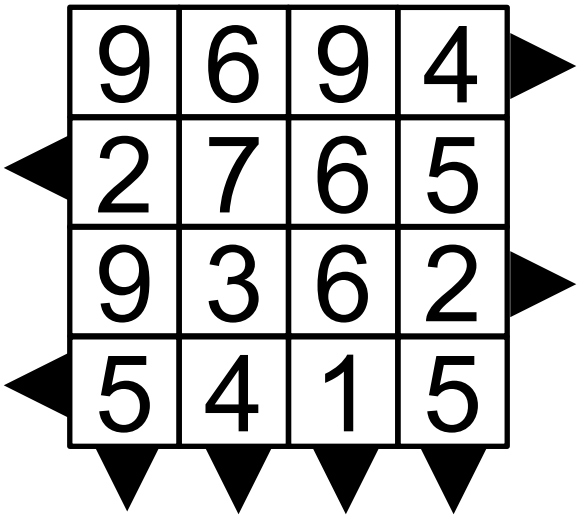
Sortează crescător coloanele

Shear sort



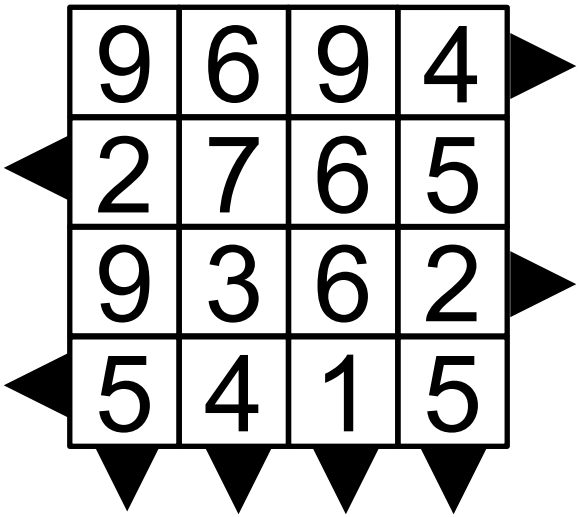
Repetă tot de $log_2 n$ ori

Shear sort



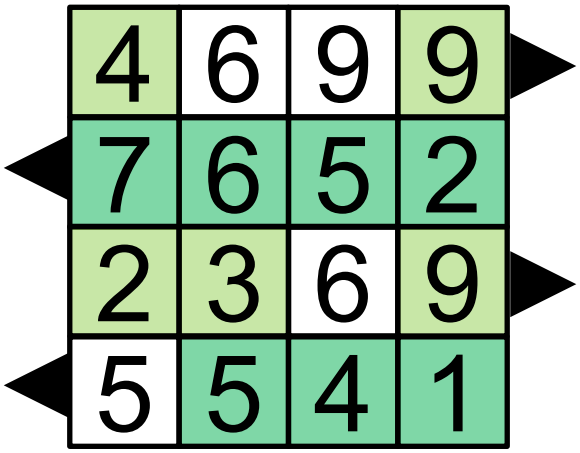
De ce sortăm liniile pare crescător și cele impare descrescător?

Shear sort

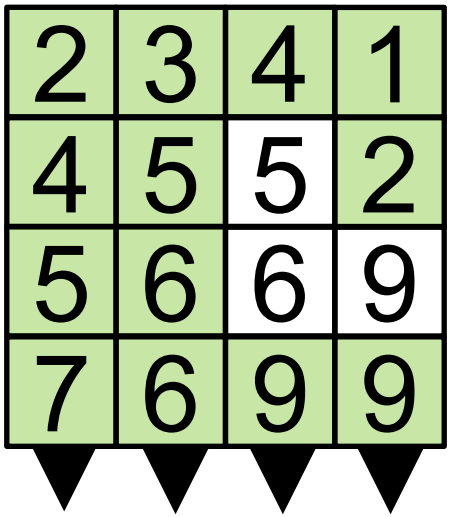


De ce liniile pare crescător și celelalte descrescător?
Dorim compararea celui mai mare element de pe linia i cu cel mai mic de pe linia i+1

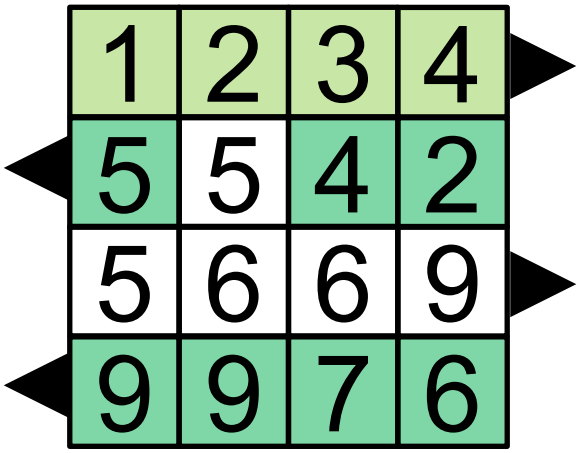
Shear sort



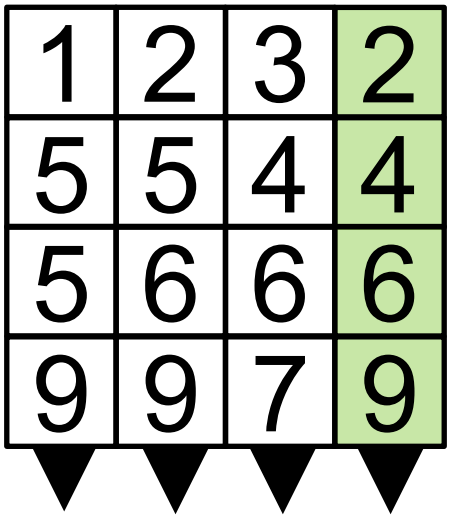
Shear sort



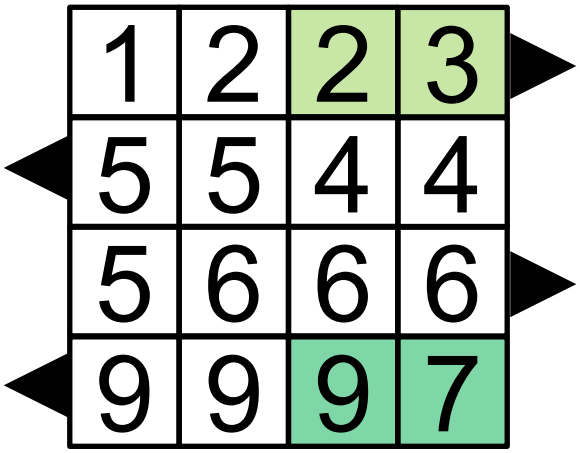
Shear sort



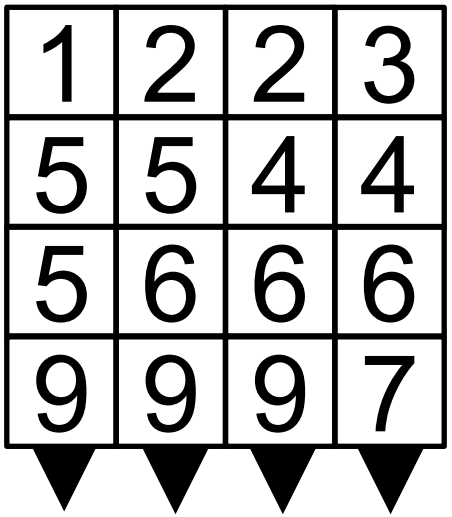
Shear sort



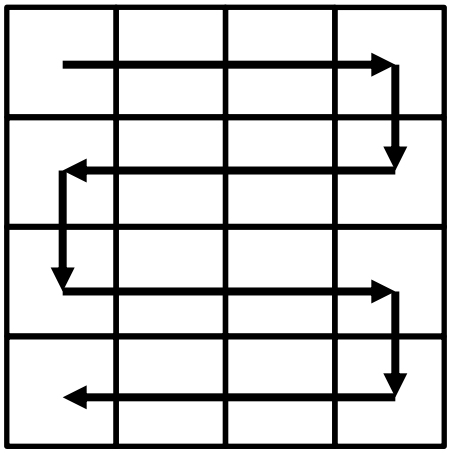
Shear sort



Shear sort

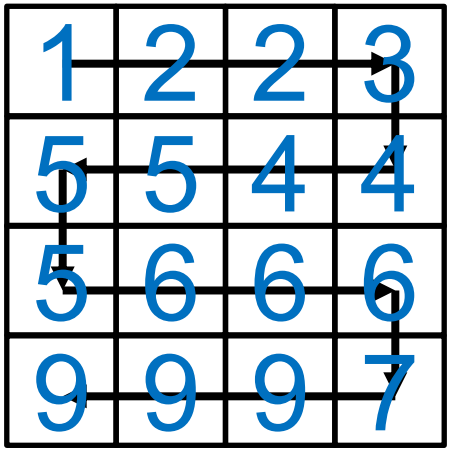


Shear sort



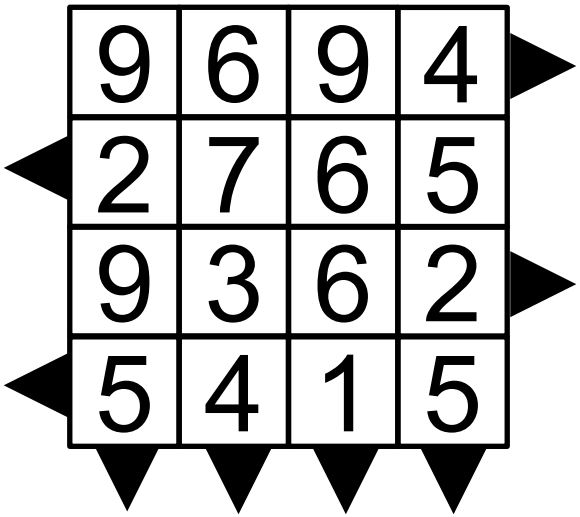
Lista finală se obține citind în formă șerpuită (snake sort)

Shear sort



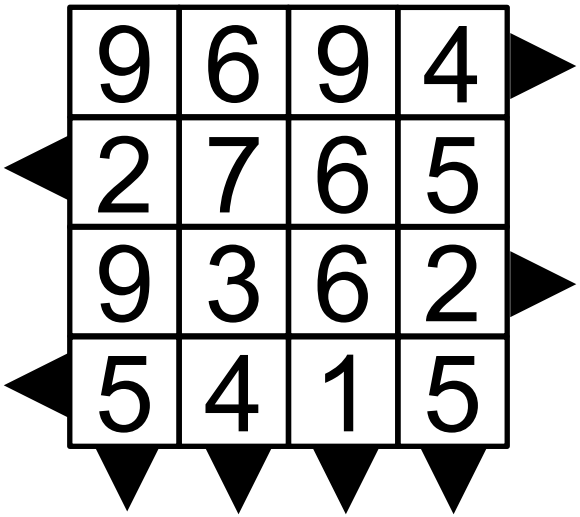
1	2	2	3	4	4	5	5	5	6	6	6	7	9	9	9

Shear sort



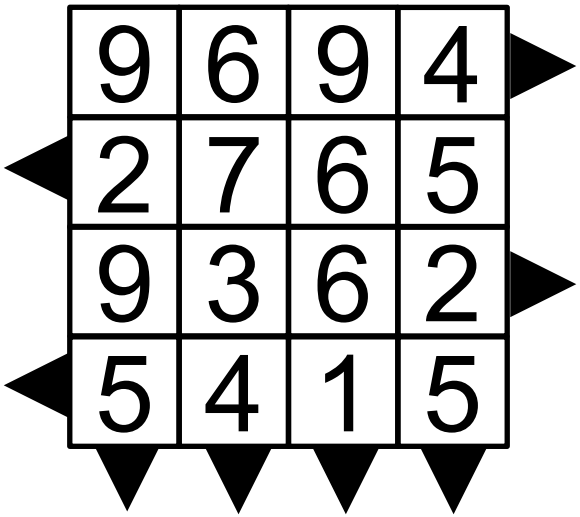
Complexitate? $G =$

Shear sort



Complexitate? $G = log_2 N * log_2 N$ repetiții

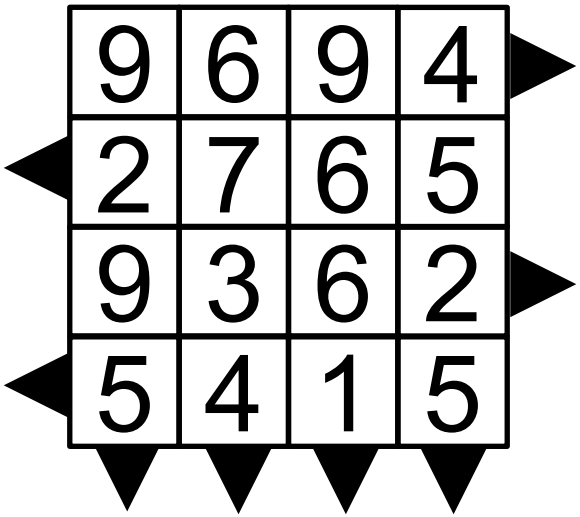
Shear sort



Complexitate? $G = log_2 N * 2 * \sqrt{N}$

$\sqrt{N}$ linii/coloane

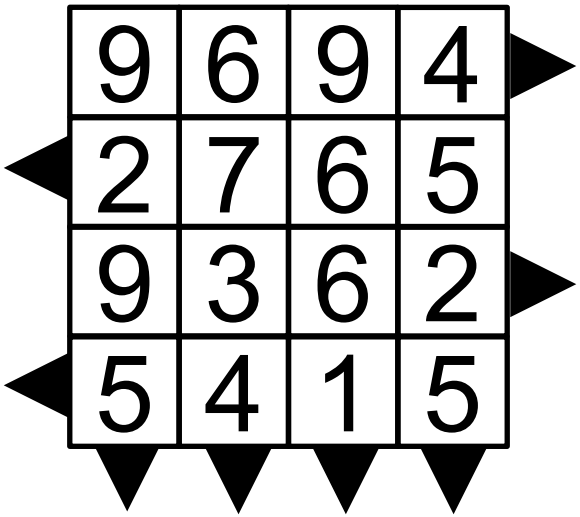
Shear sort



Complexitate? $G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$

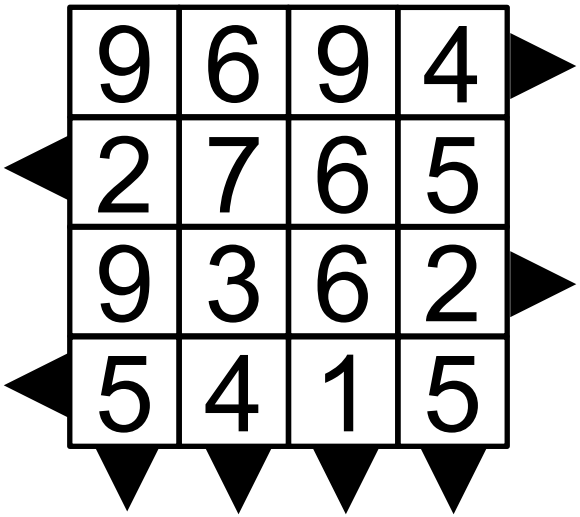
$\sqrt{N} * log_2 \sqrt{N}$ cel mai bun algoritm secvențial de sortare

Shear sort



Complexitate? $G = log_2 N * N * log_2 \sqrt{N}$

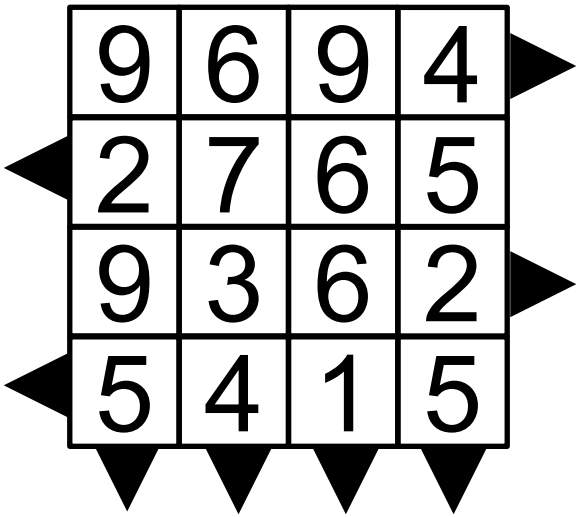
Shear sort



Complexitate? $G = Nlog_2N * log_2\sqrt{N}$

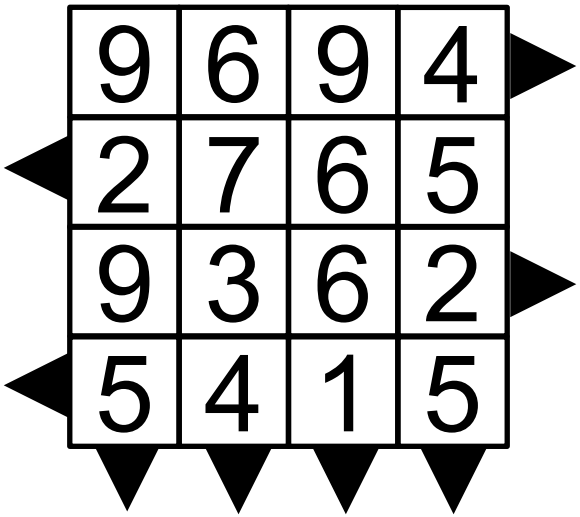
Cel mai bun algoritm secvențial rămâne $Nlog_2N$

Shear sort



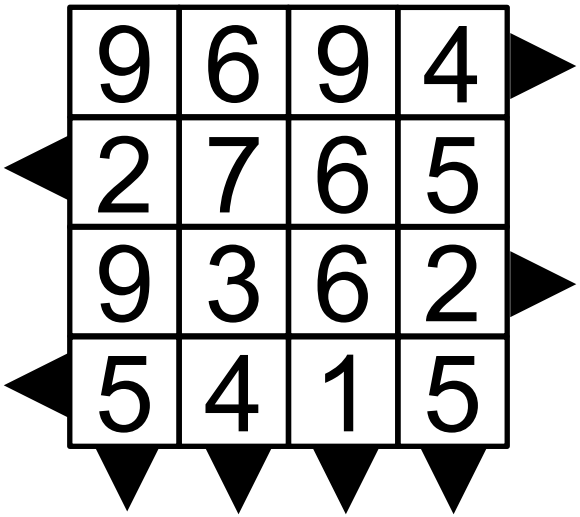
Cum paralelizăm?

Shear sort



Cum paralelizăm?
Toate liniile în paralel, barieră, apoi toate coloanele, apoi barieră, și repetăm.

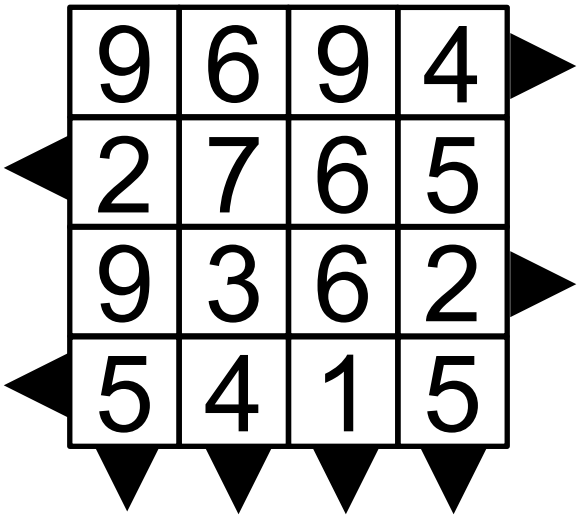
Shear sort



Complexitate versiune paralelă?

$$G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$$

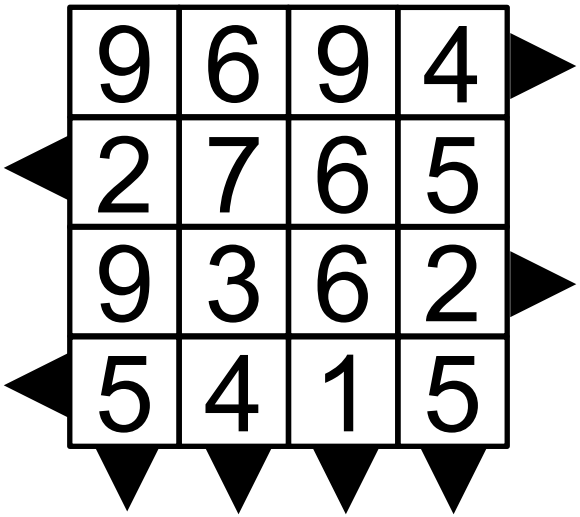
Shear sort



Complexitate versiune paralelă? N = P

$$T = log_2 N * 2 * \sqrt{N} * log_2 \sqrt{N}$$

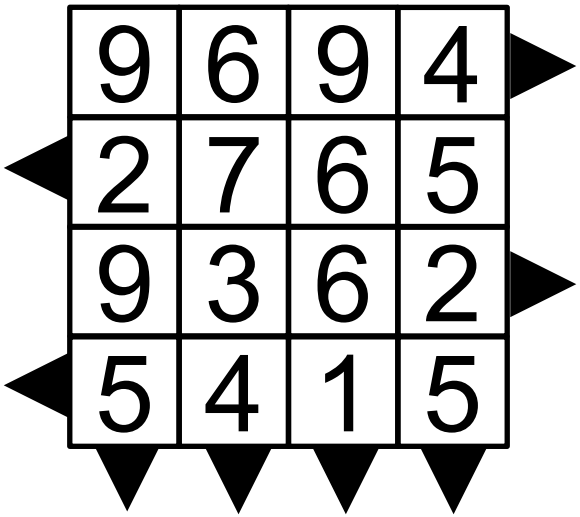
Shear sort



Complexitate versiune paralelă? N = P

$$T = \sqrt{N}\, log_2 \sqrt{N} * log_2 N$$

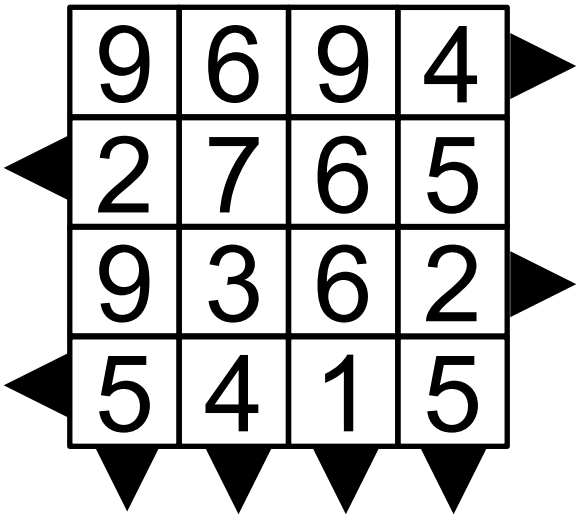
Shear sort



Complexitate versiune paralelă?

$$T = log_2 N * 2 * \frac{\sqrt{N}}{P} * \sqrt{N} * log_2 \sqrt{N}$$

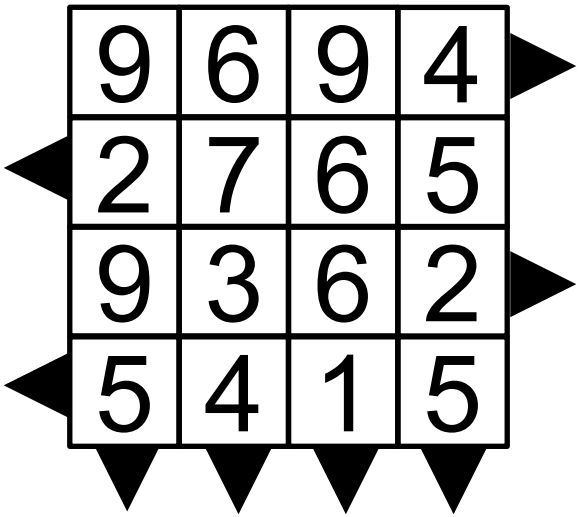
Shear sort



Complexitate versiune paralelă?

$$T = \frac{N}{P} log_2 N * log_2 \sqrt{N}$$

Shear sort



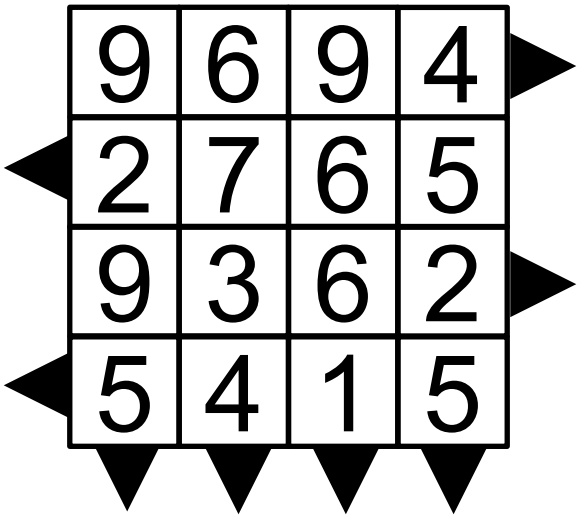
Speedup?

$T = \frac{N}{P} log_2 N \ * \ log_2 \sqrt{N}$

$G = \ N log_2 N$

Shear sort

$$S = \frac{Nlog_2N}{\frac{N}{P}log_2N \ * \ log_2\sqrt{N}}$$



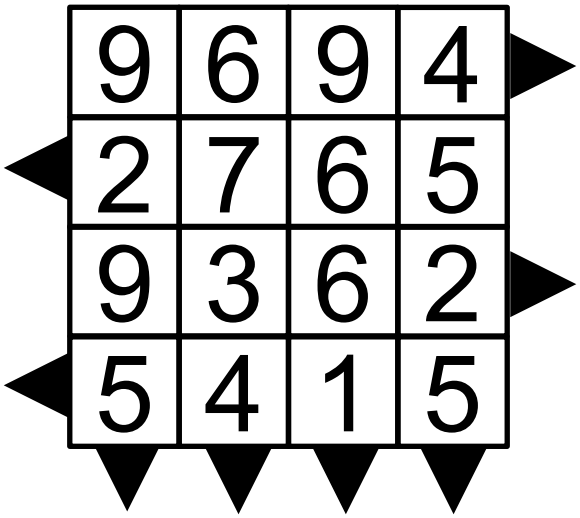
Speedup?

$$T = \frac{N}{P}log_2N \ * \ log_2\sqrt{N}$$

$$G = \ Nlog_2N$$

Shear sort

$$S = \frac{P}{log_2\sqrt{N}}$$



Speedup?

$$T = \frac{N}{P} log_2 N \ * \ log_2\sqrt{N}$$

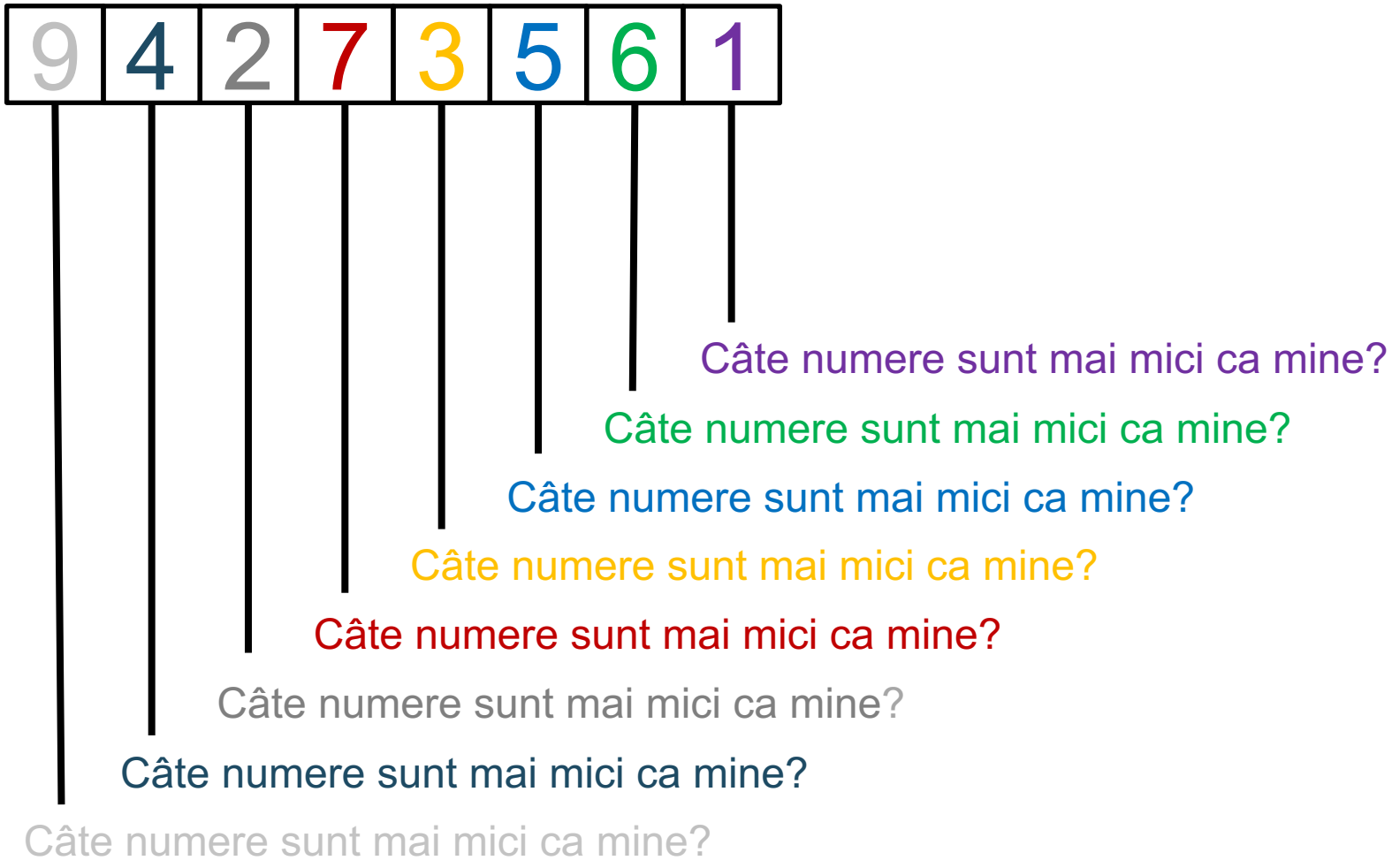
$$G = \ N log_2 N$$

Rank Sort

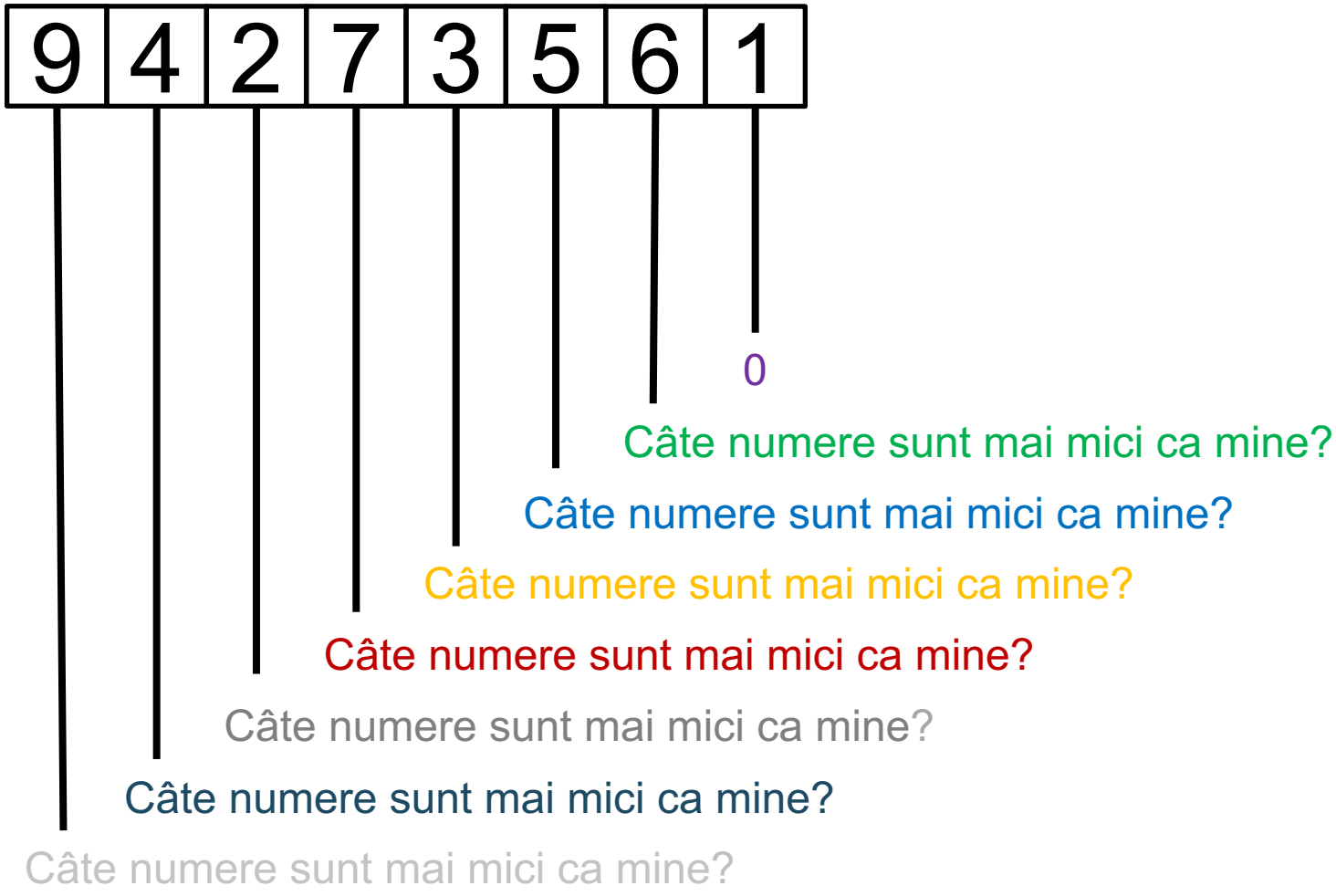
9	4	2	7	6	5	6	1

1	2	4	5	6	6	7	9

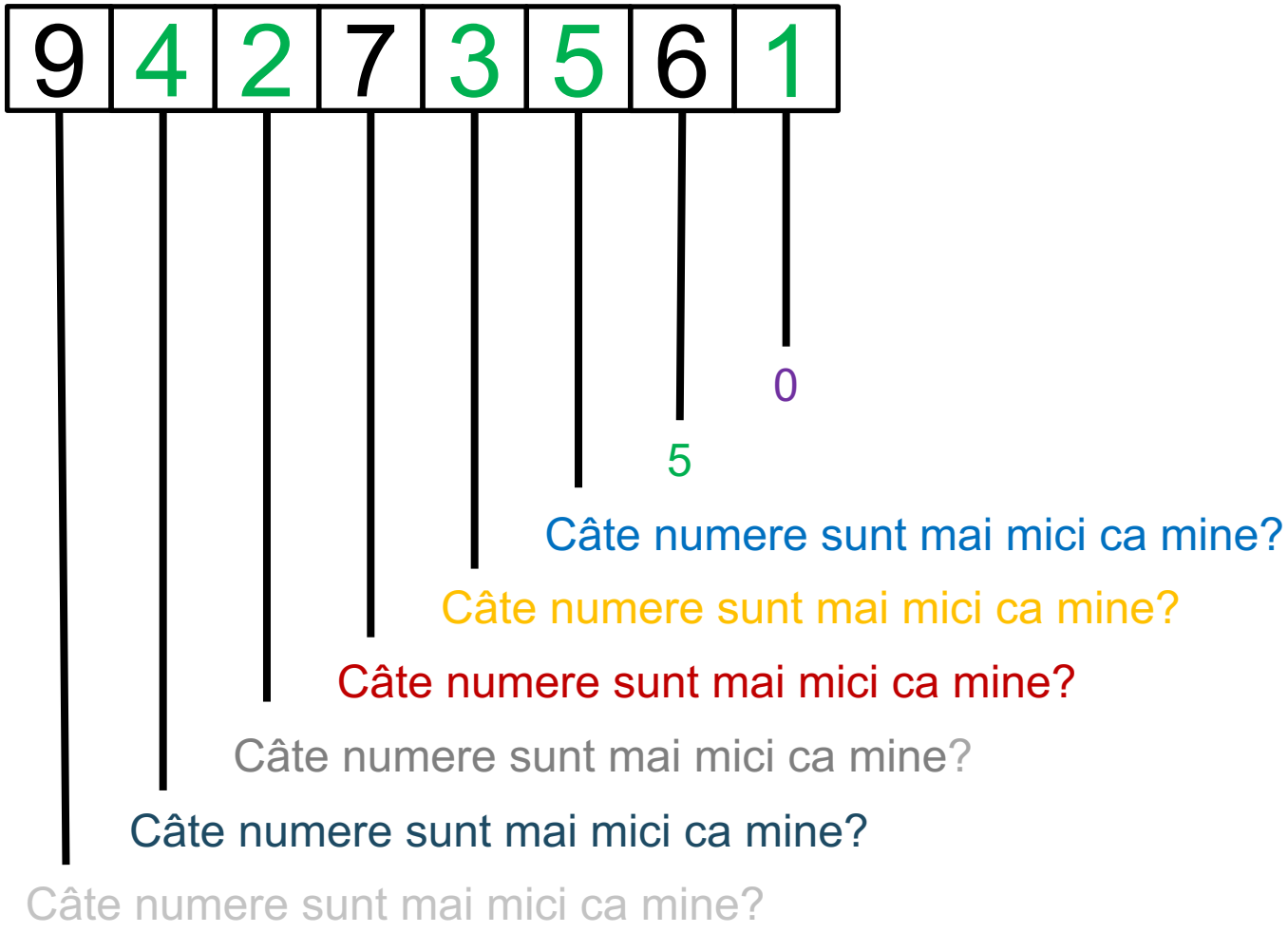
Rank Sort


9 4 2 7 3 5 6 1
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

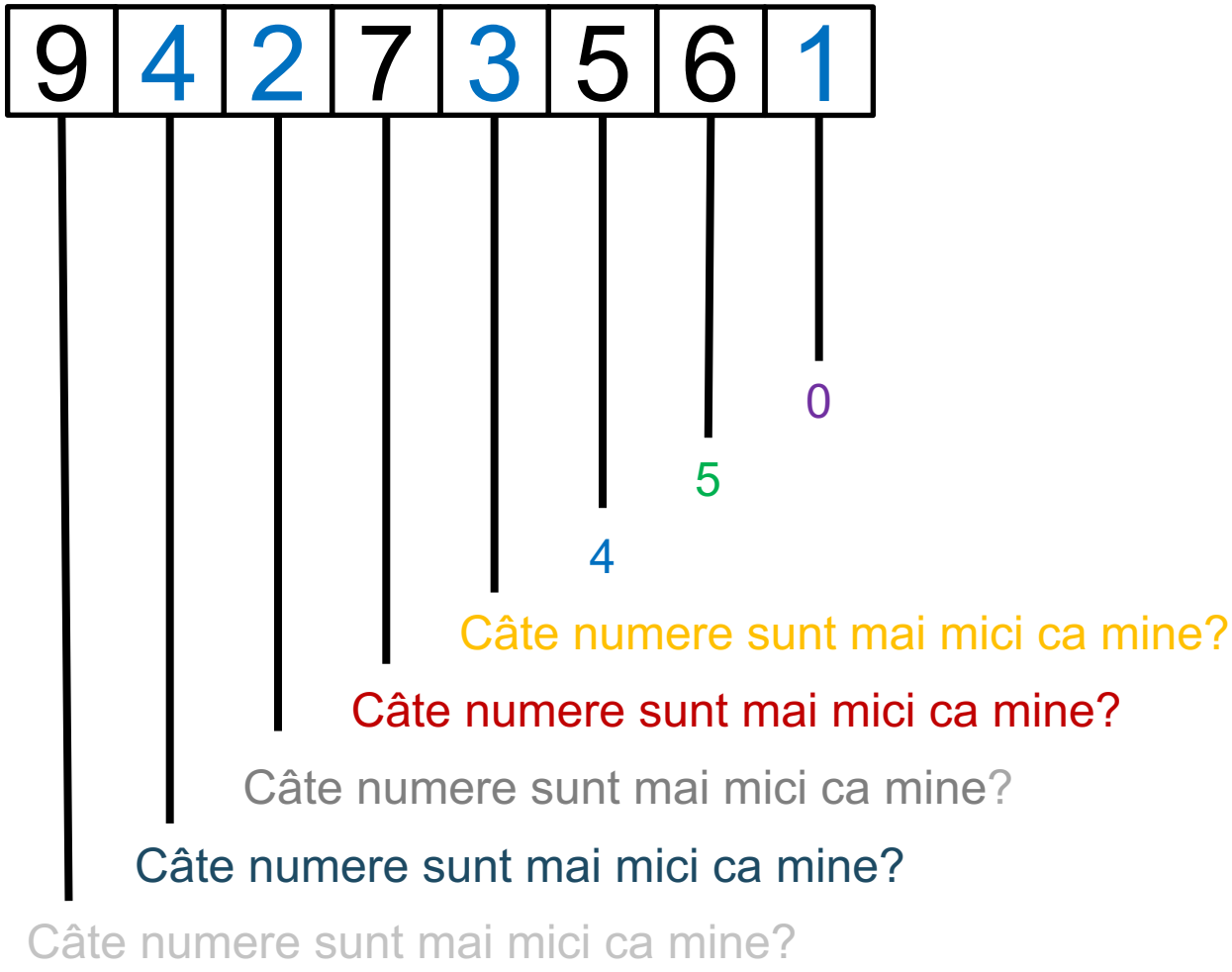
Rank Sort


9 4 2 7 3 5 6 1
0
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

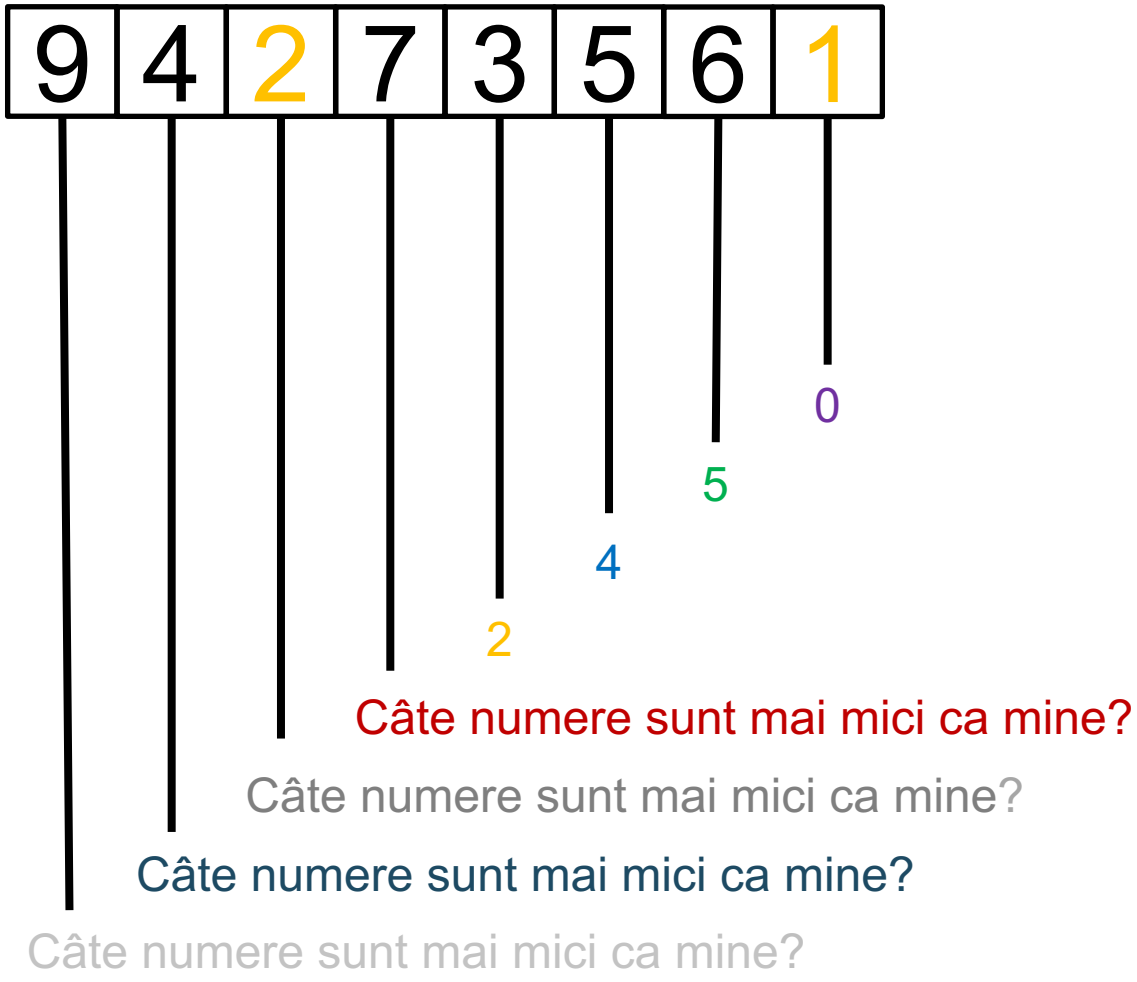
Rank Sort


9 4 2 7 3 5 6 1
0
5
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

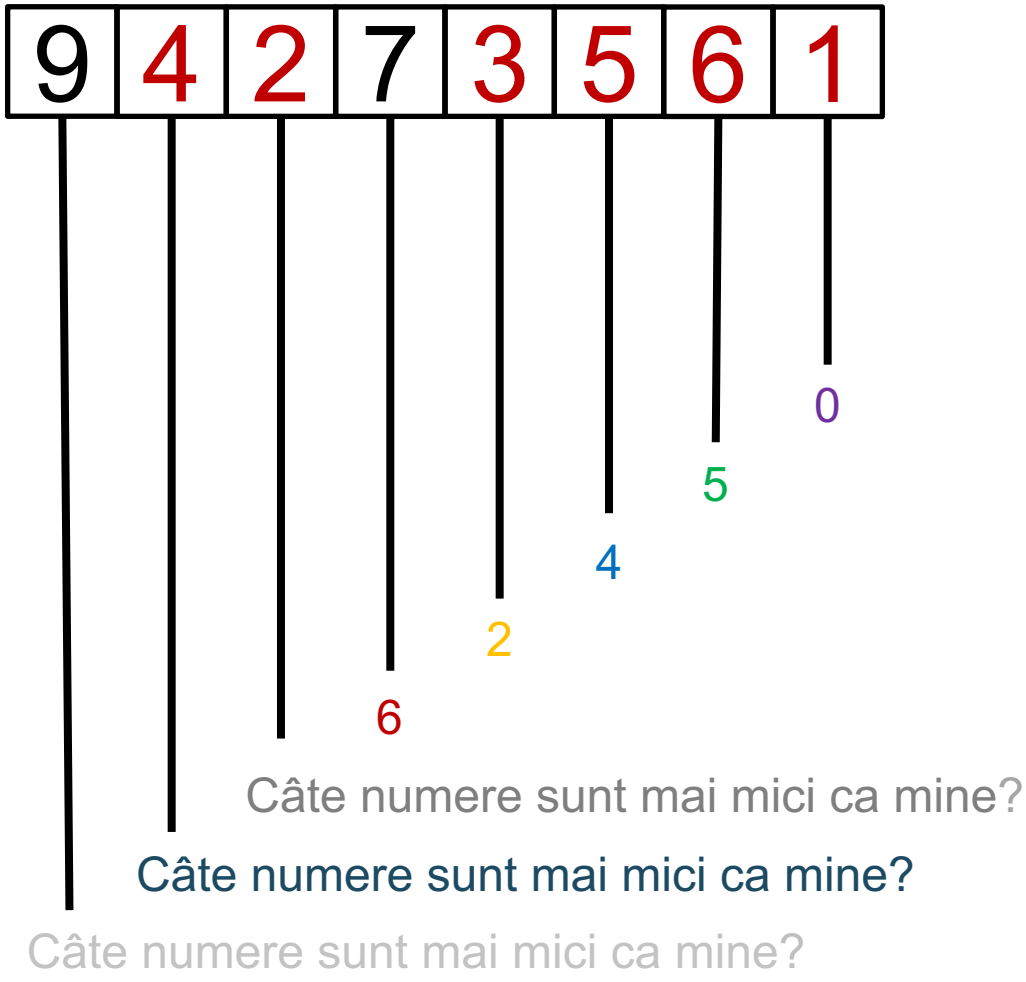
Rank Sort


9 4 2 7 3 5 6 1
0
5
4
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

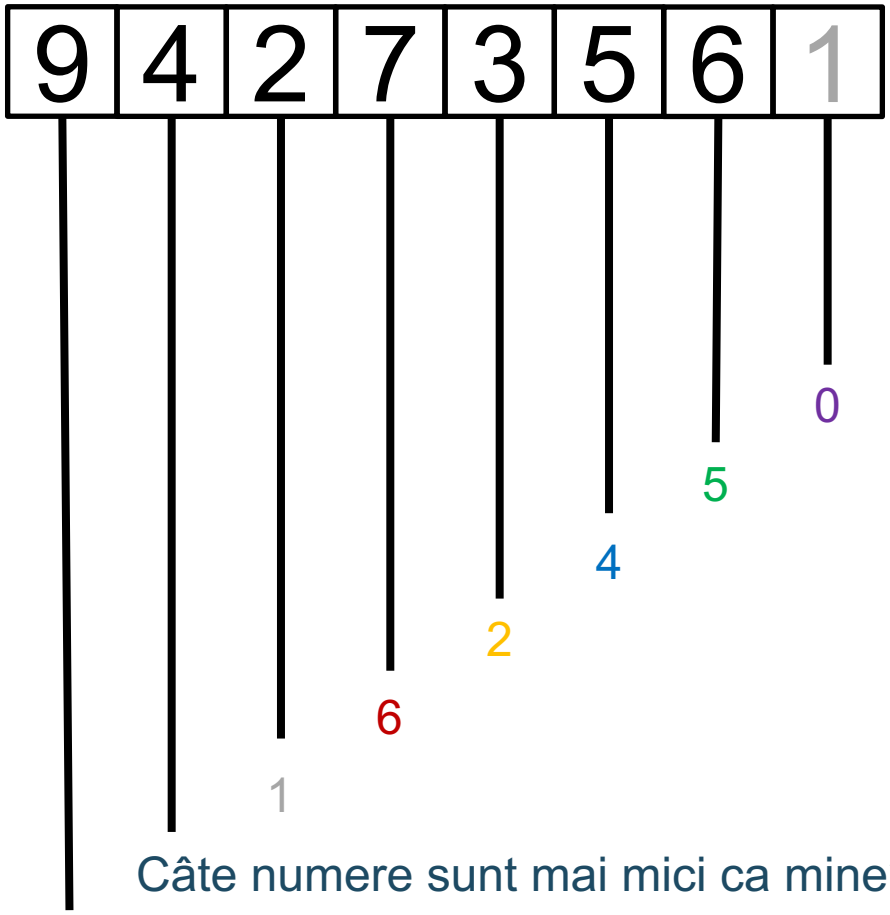
Rank Sort


9 4 2 7 3 5 6 1
0
5
4
2
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

Rank Sort

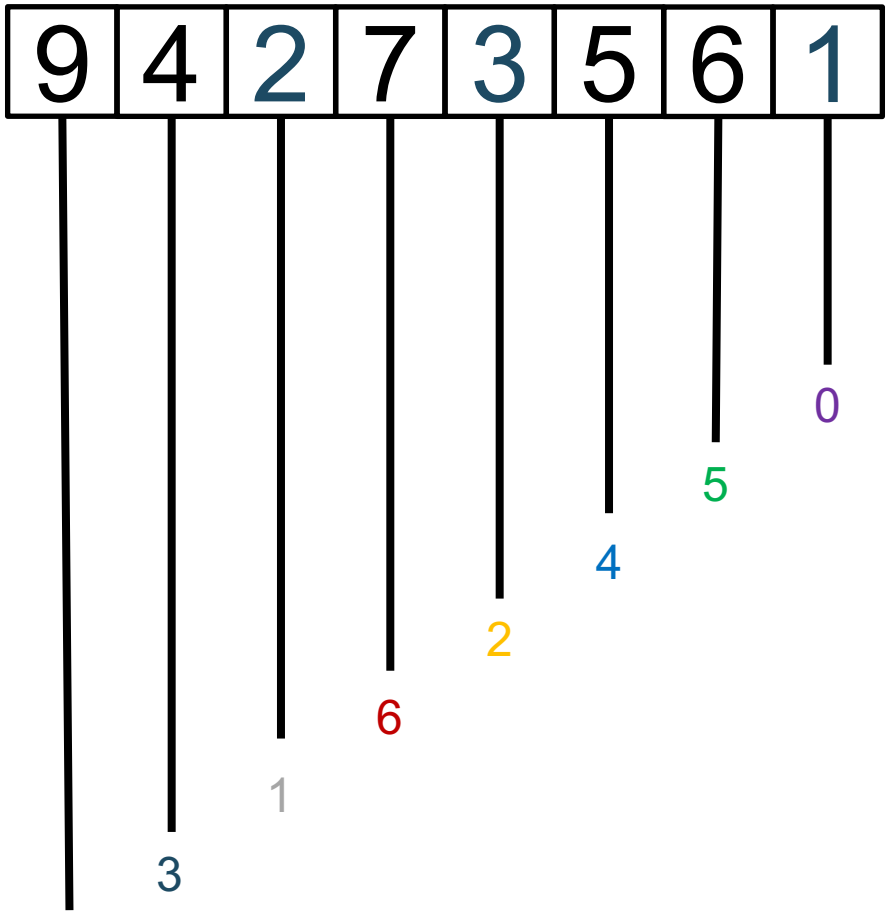

9 4 2 7 3 5 6 1
6
2
4
5
0
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

Rank Sort



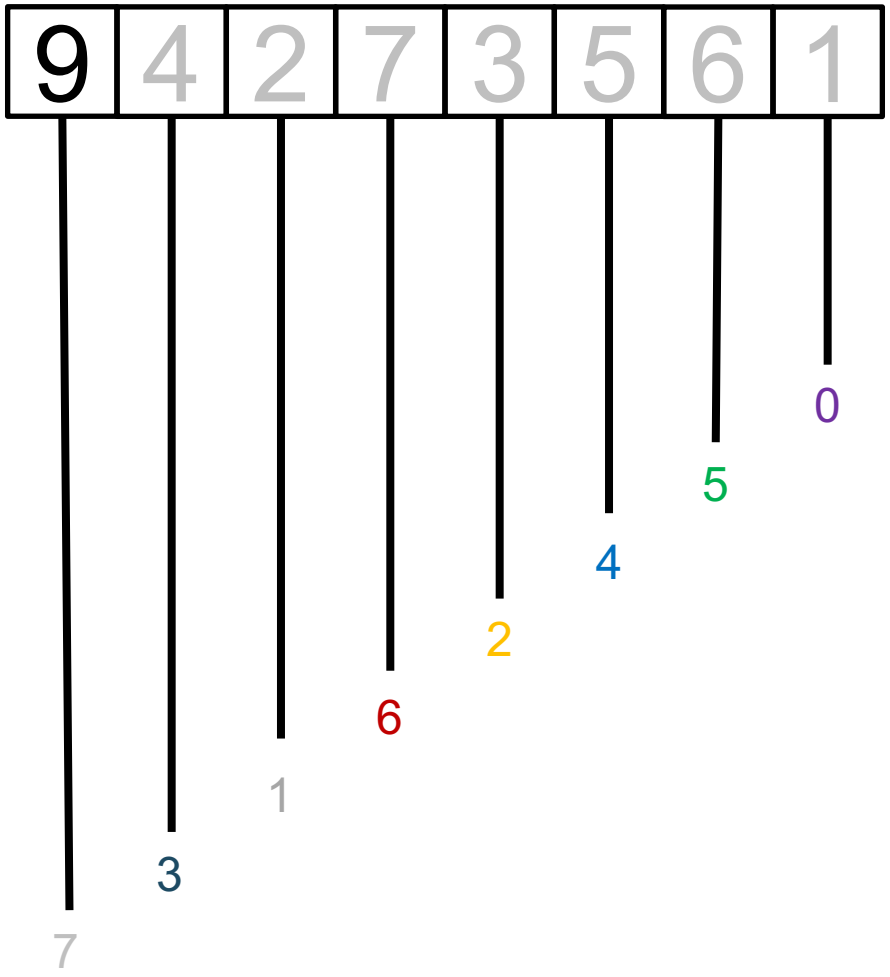
Câte numere sunt mai mici ca mine?

Rank Sort

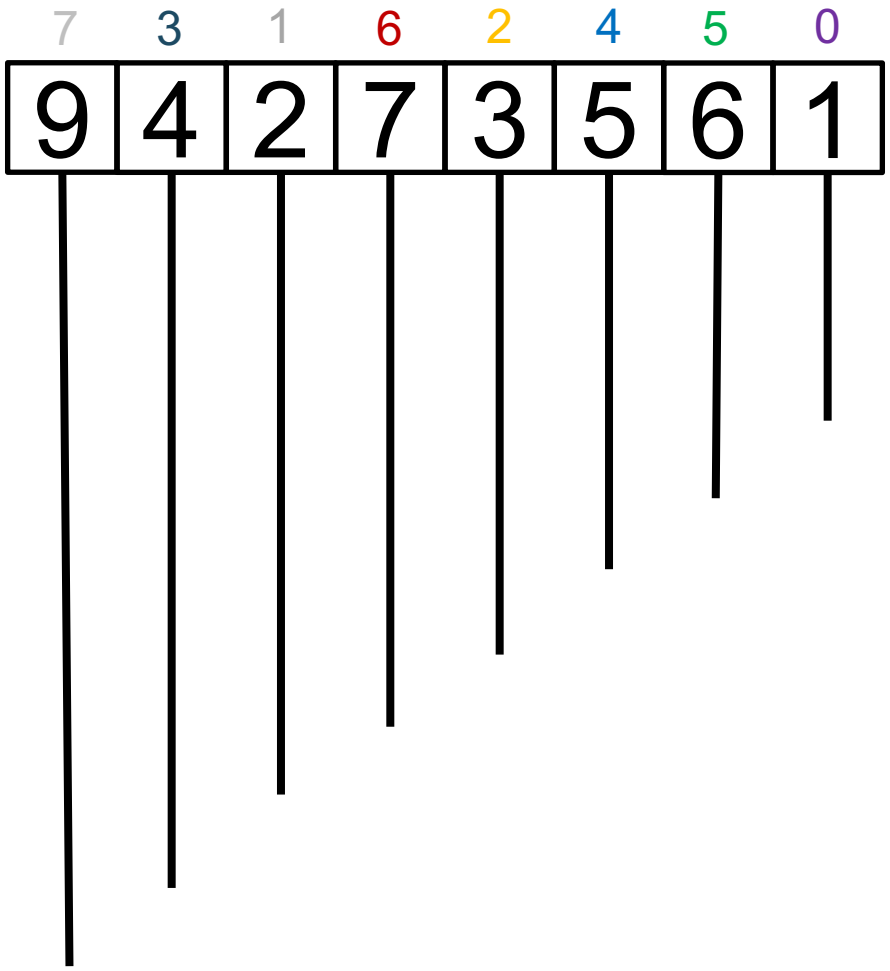


Câte numere sunt mai mici ca mine?

Rank Sort

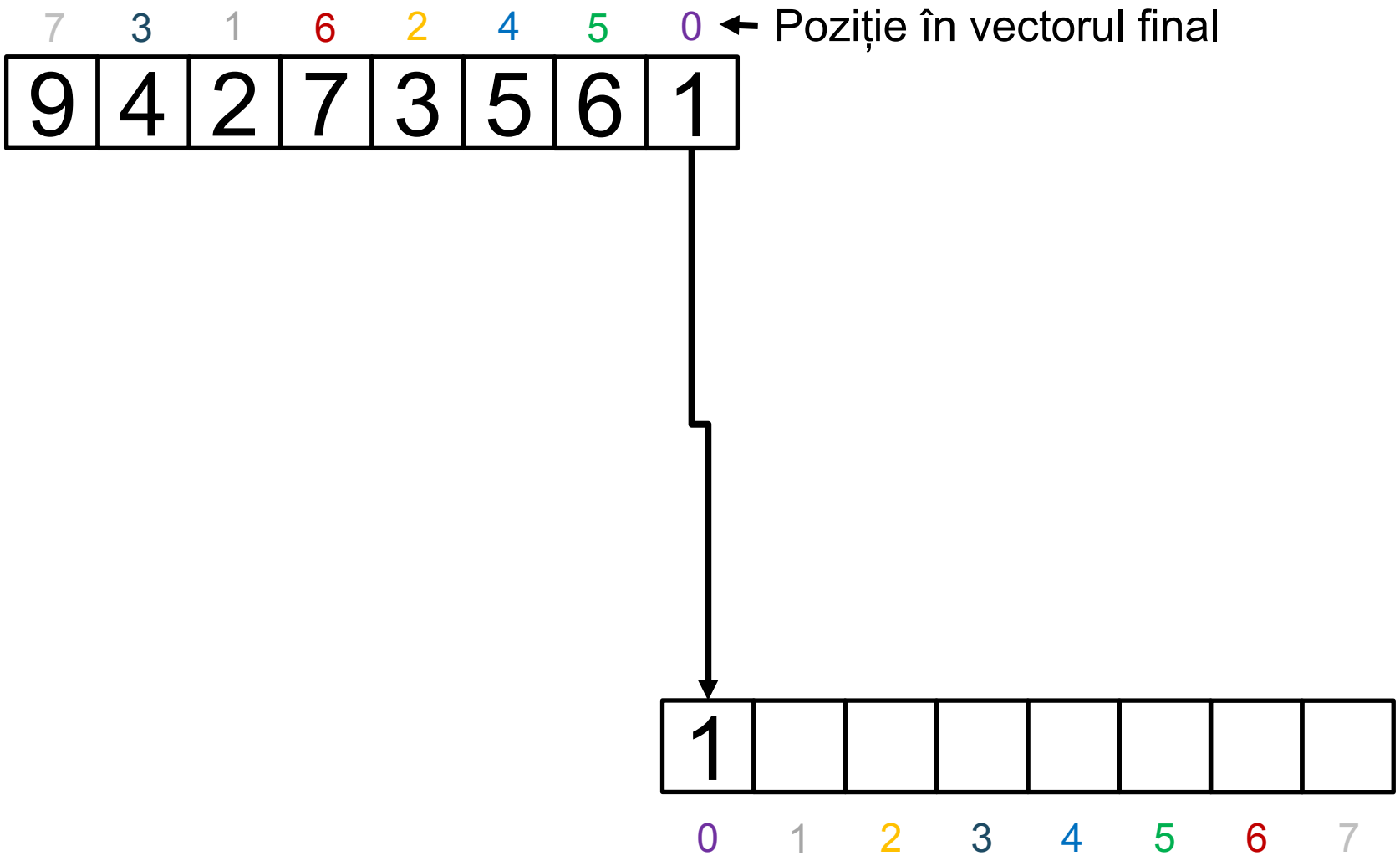


Rank Sort

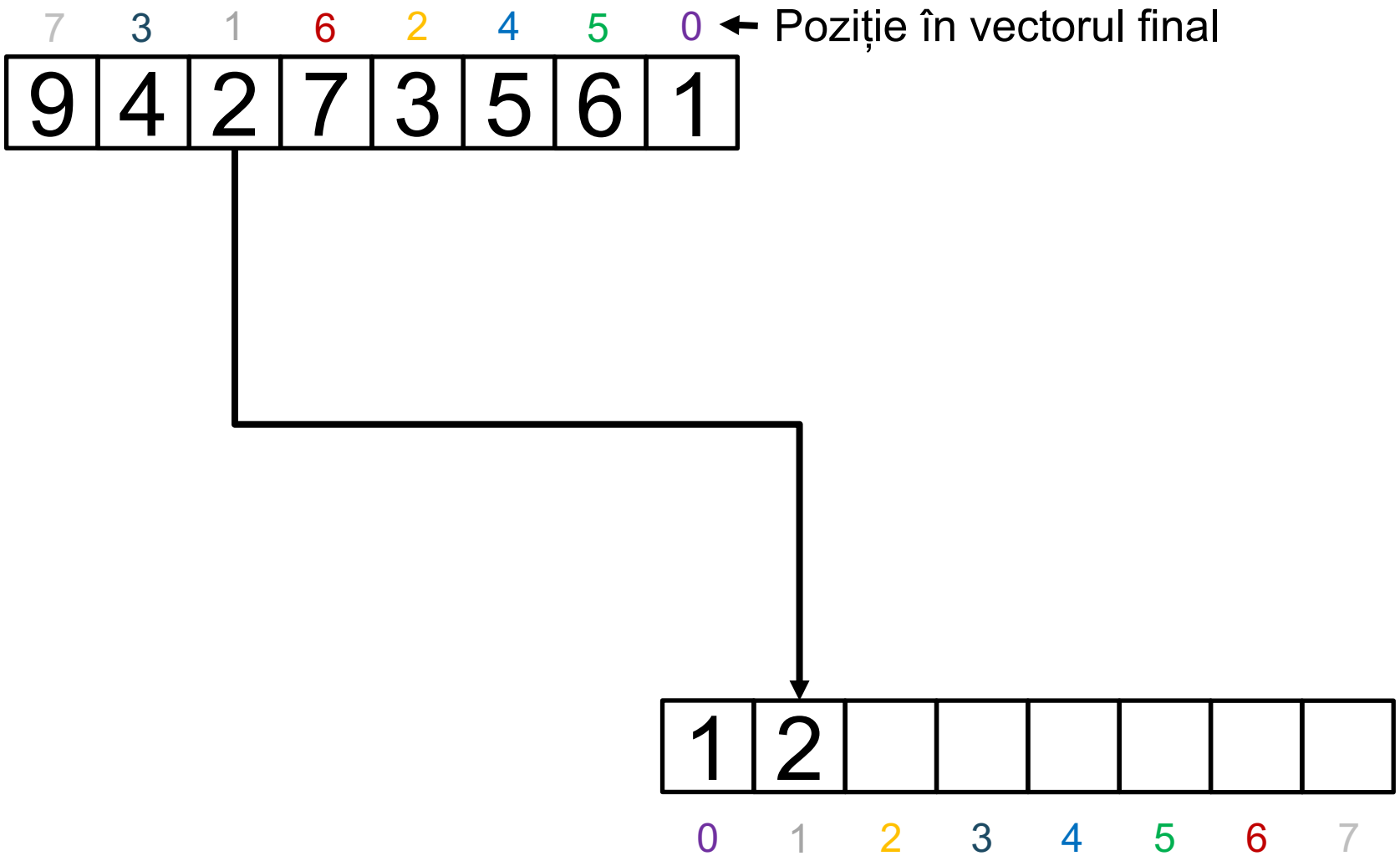


Poziții în vectorul final

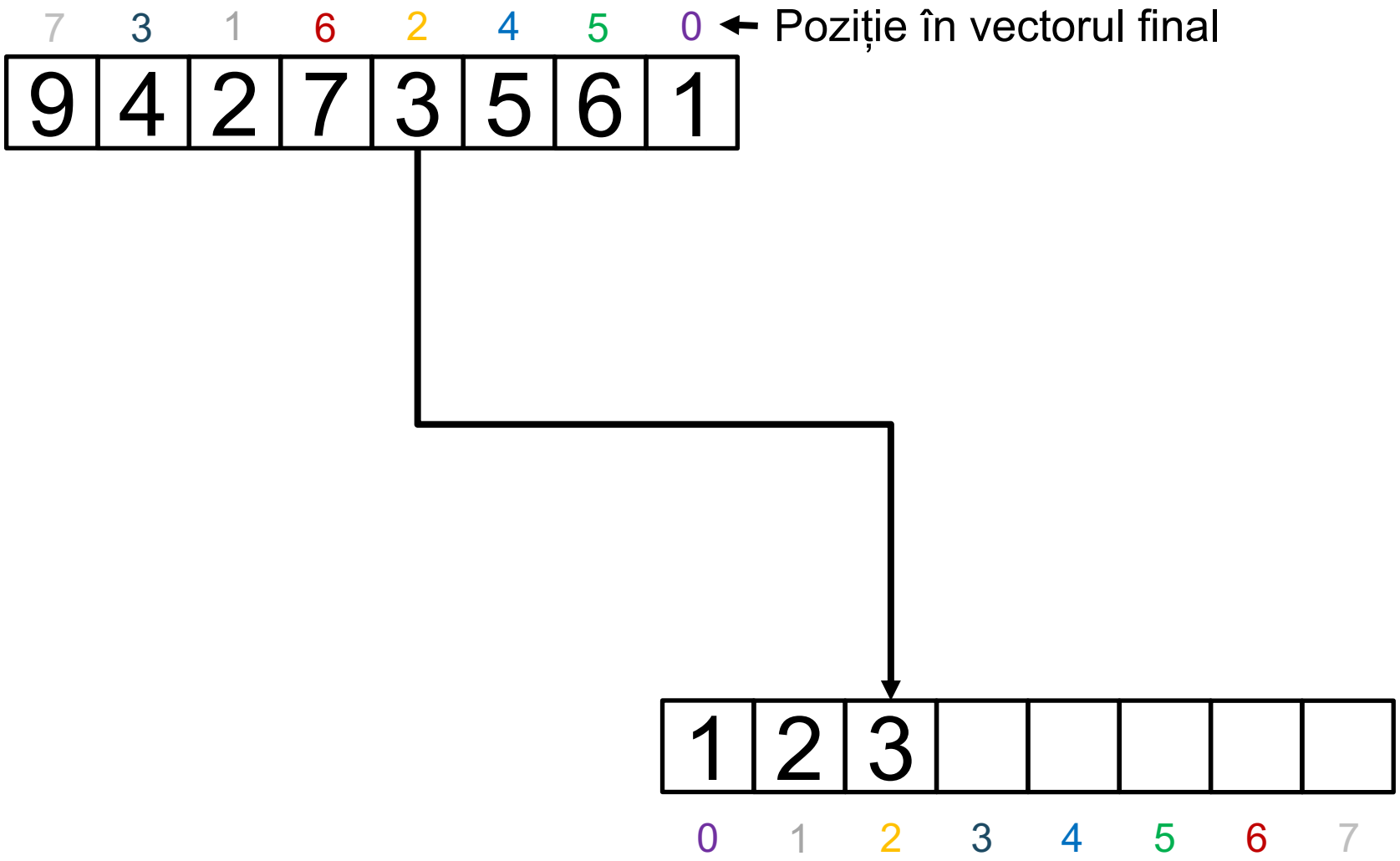
Rank Sort



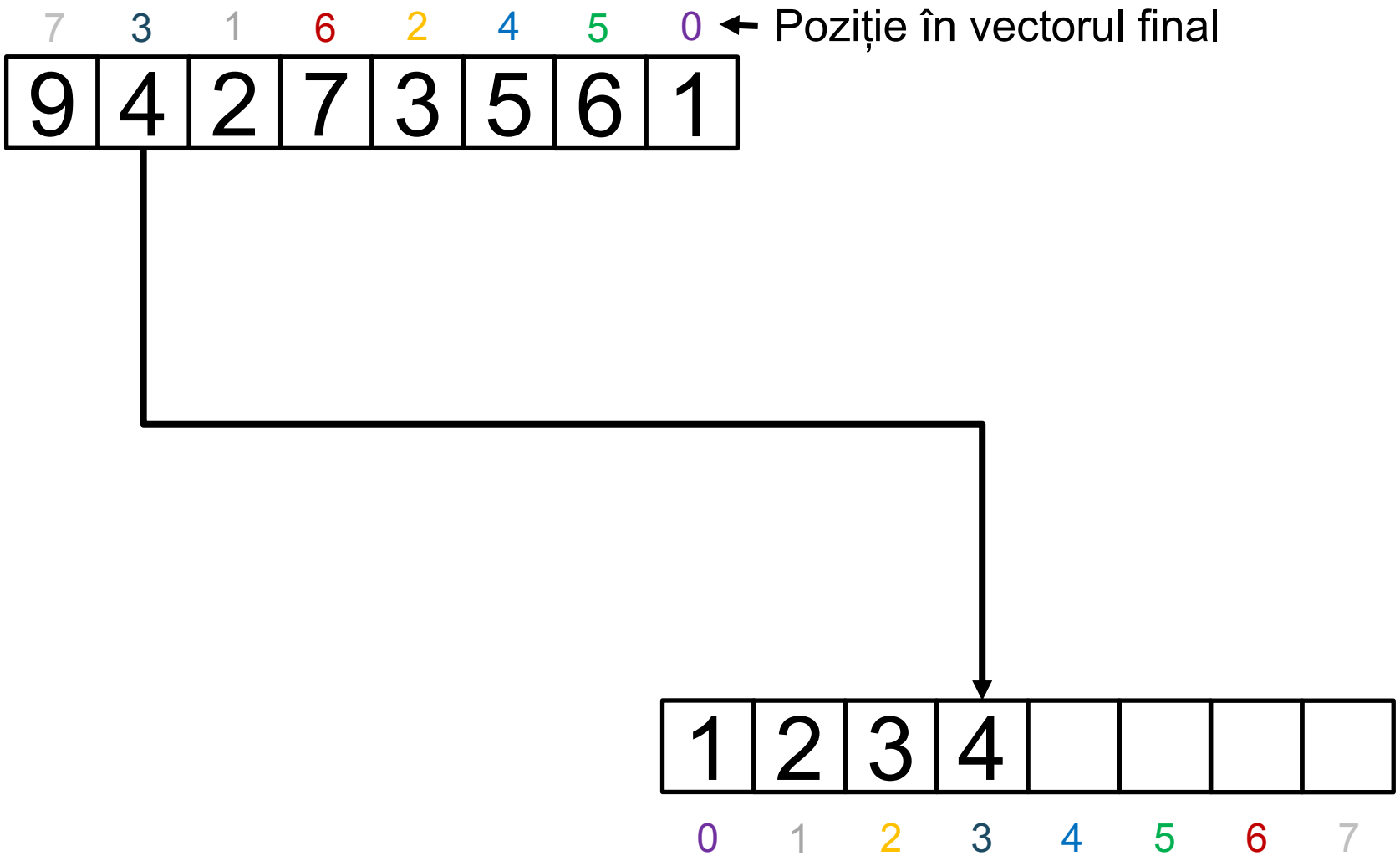
Rank Sort



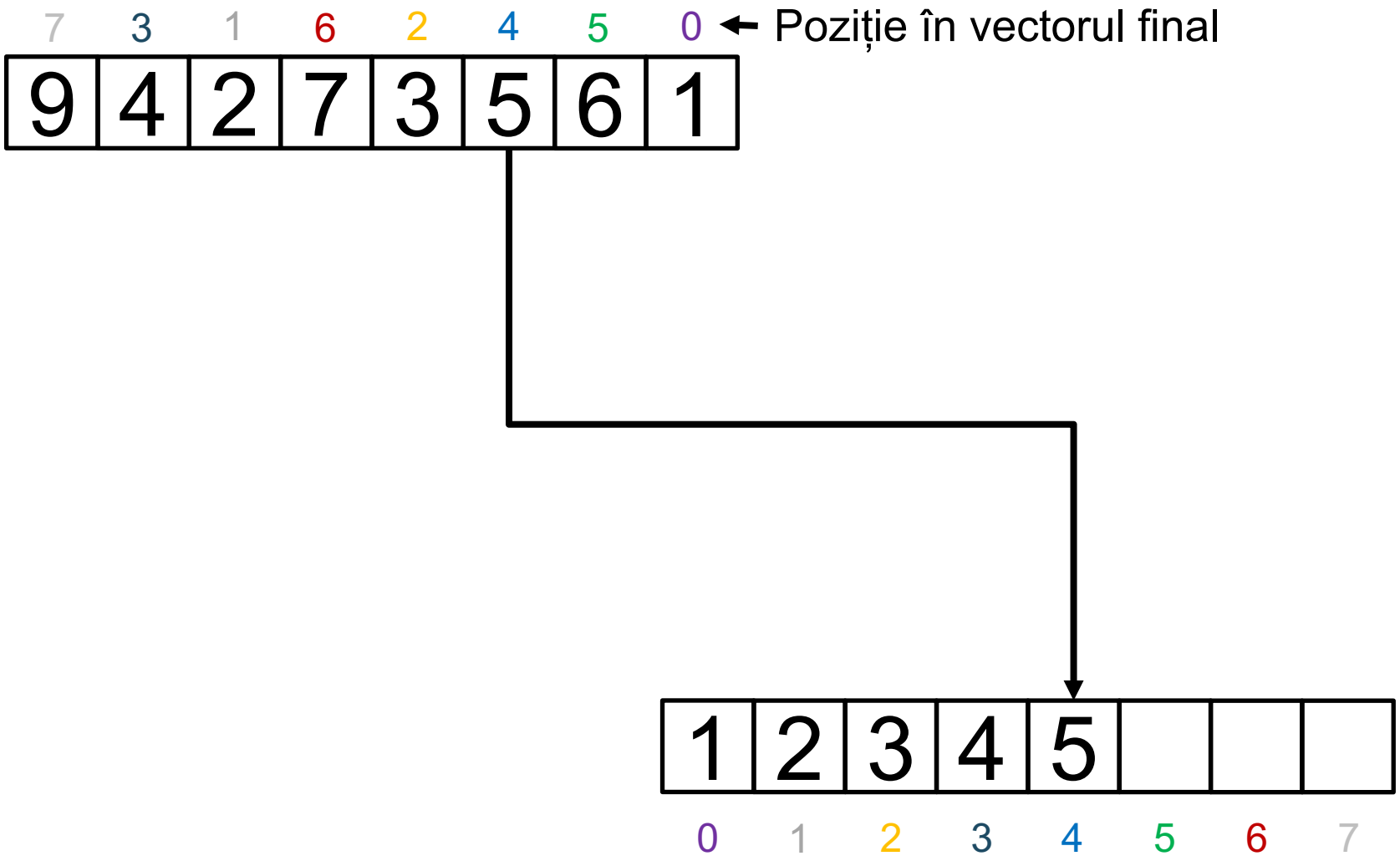
Rank Sort



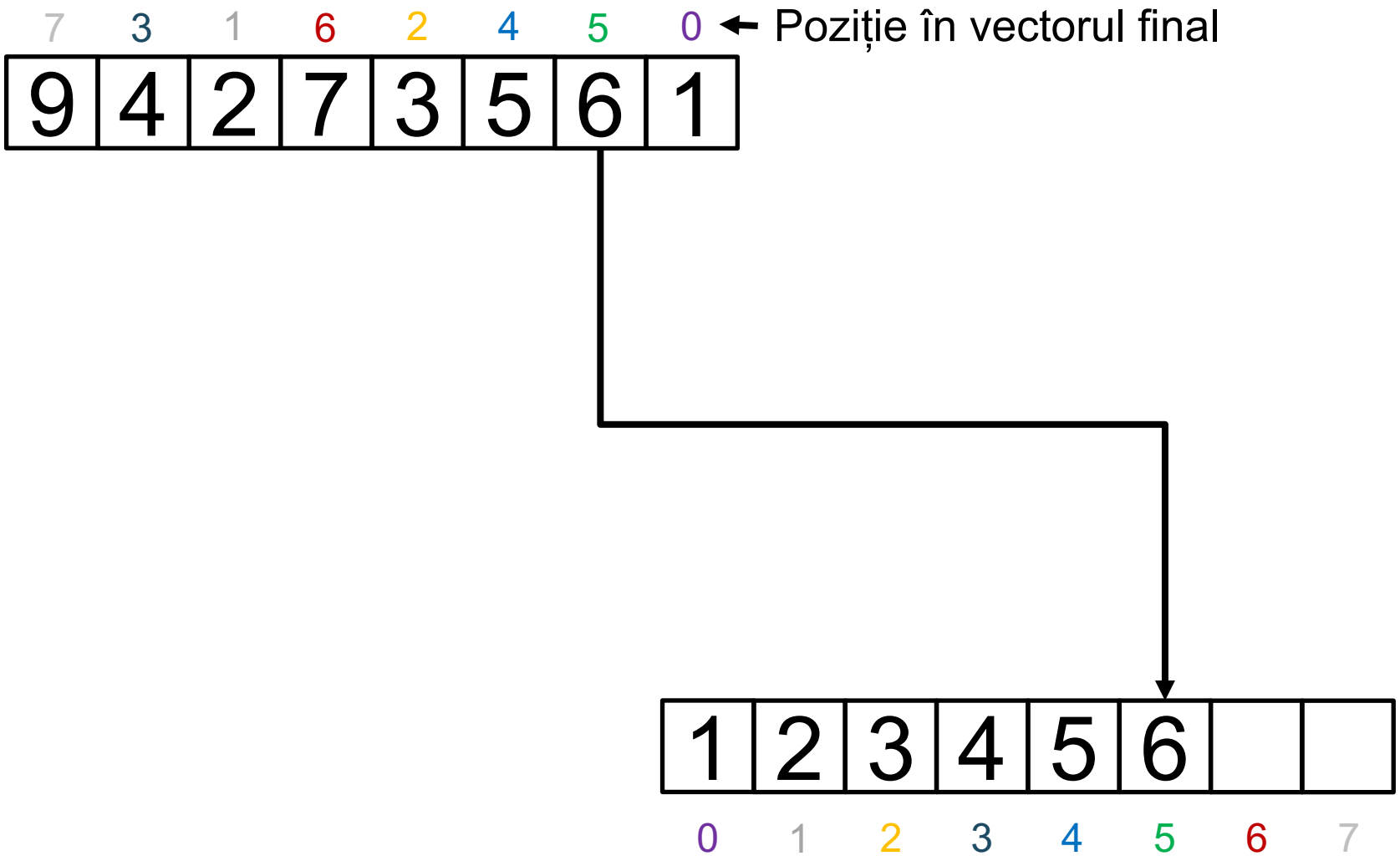
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4
0 1 2 3 4 5 6 7

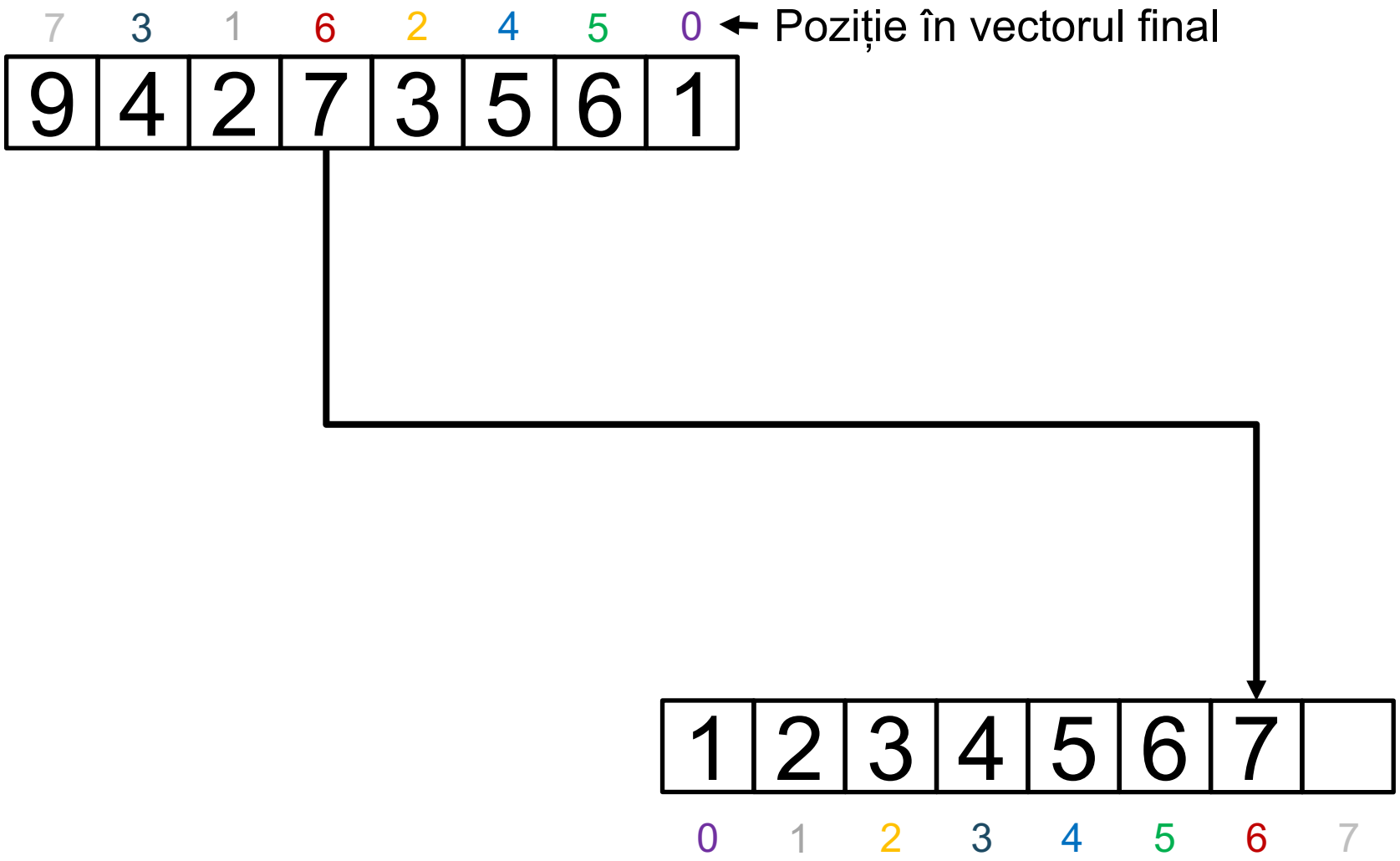
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5
0 1 2 3 4 5 6 7

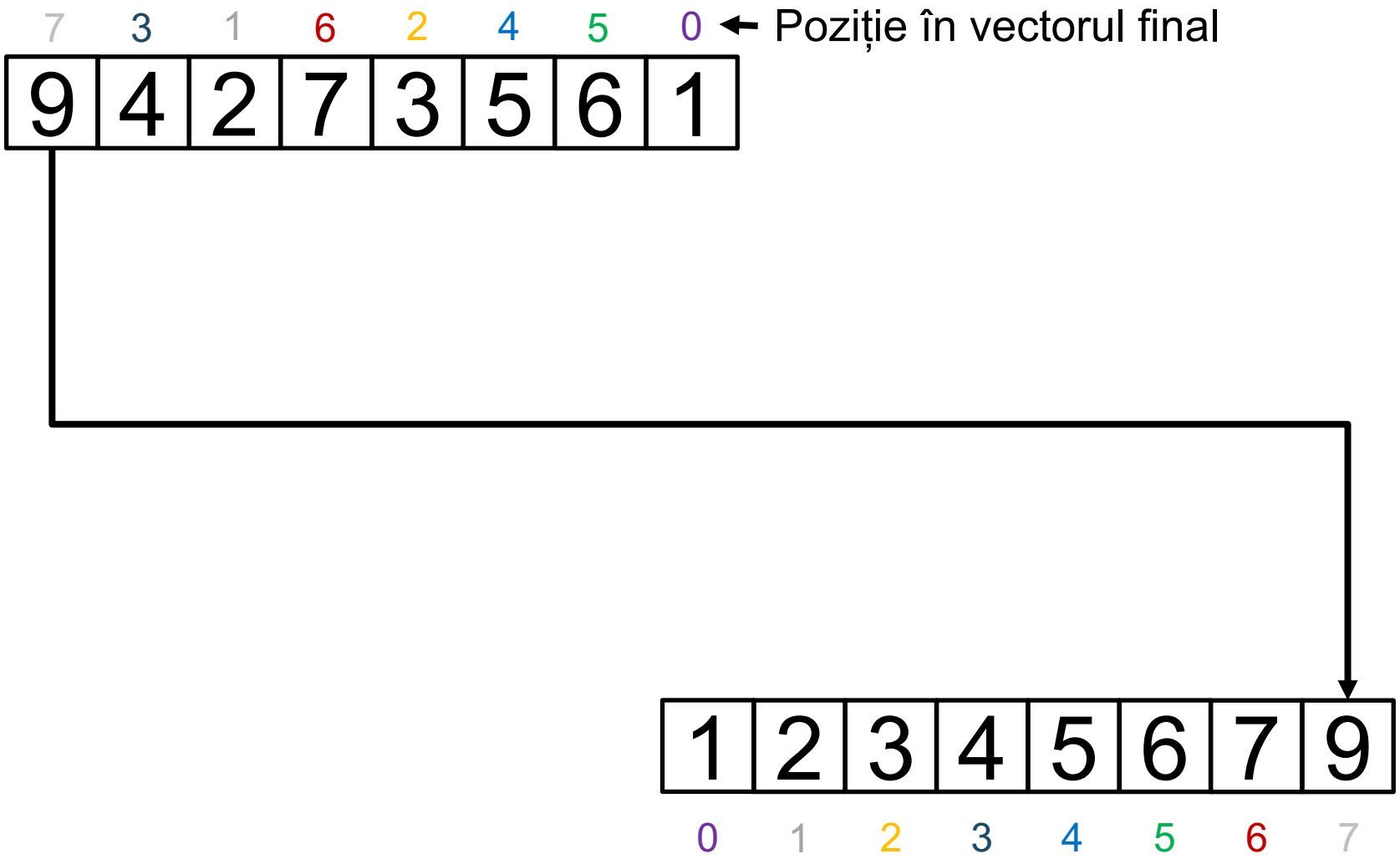
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6
0 1 2 3 4 5 6 7

Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6 7
0 1 2 3 4 5 6 7

Rank Sort

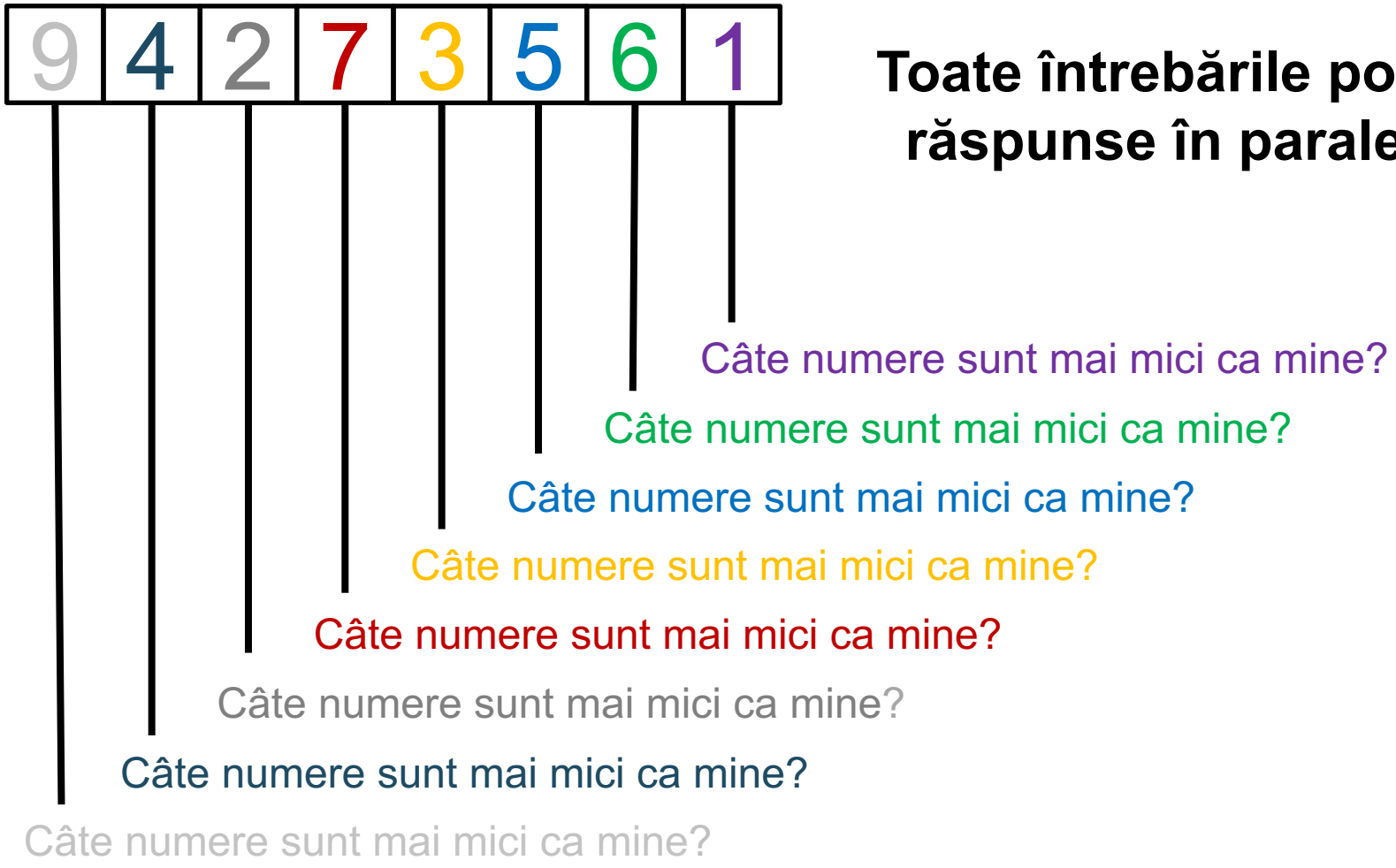

7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6 7 9
0 1 2 3 4 5 6 7

Parallel Rank Sort

9	4	2	7	6	5	6	1

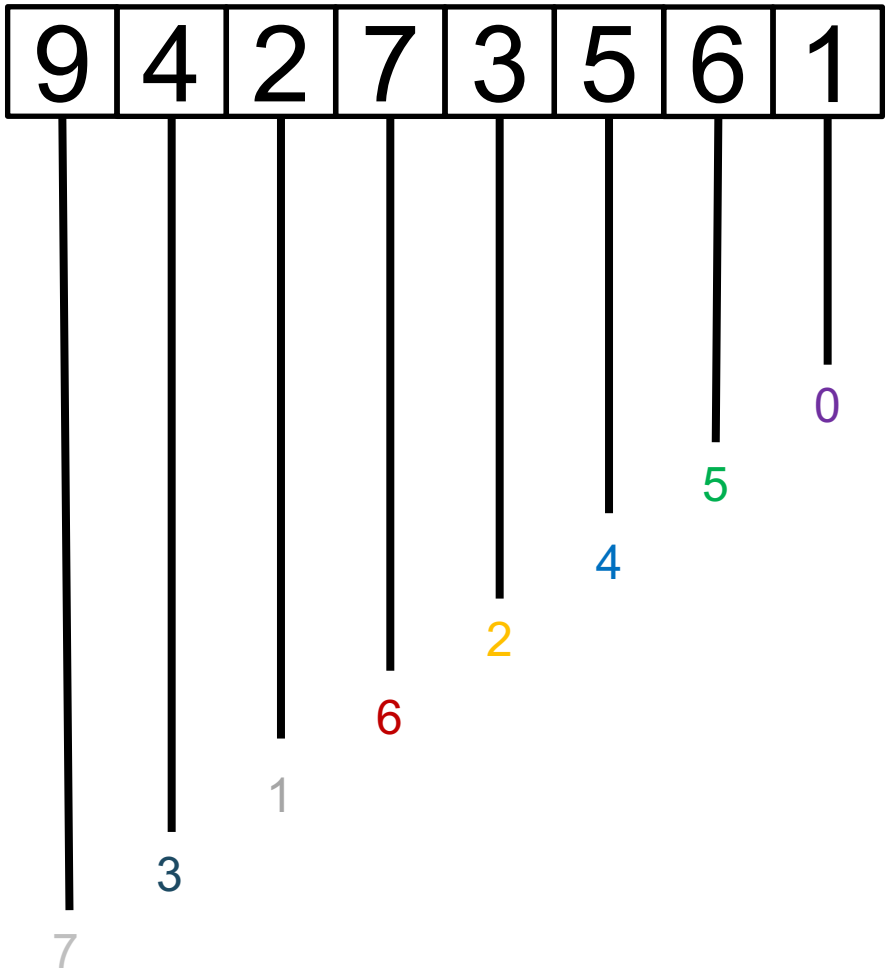
1	2	4	5	6	6	7	9

Parallel Rank Sort



Toate întrebările pot fi răspunse în paralel.

Parallel Rank Sort

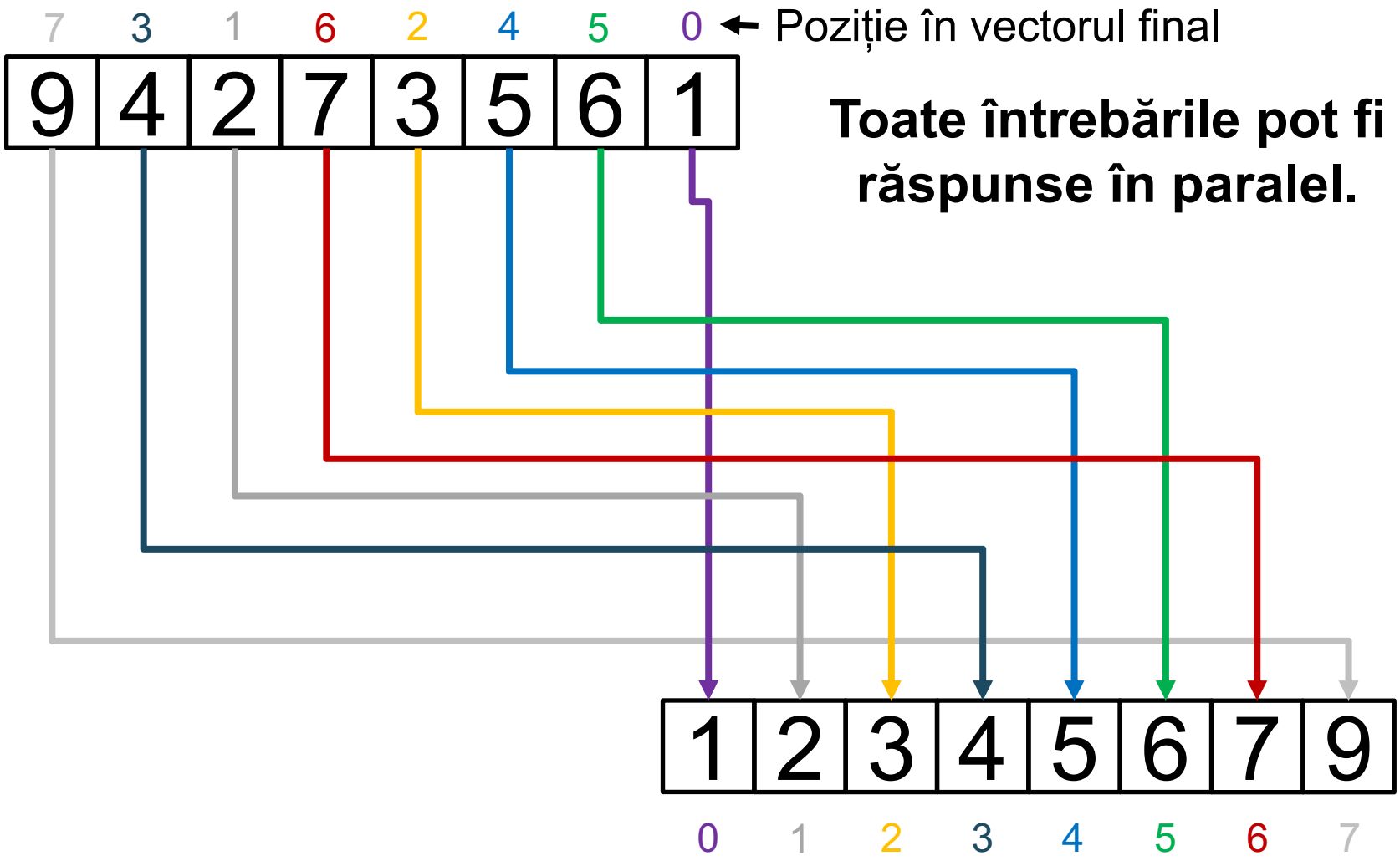


Toate întrebările pot fi răspunse în paralel.

Parallel Rank Sort



Parallel Rank Sort



Toate întrebările pot fi răspunse în paralel.

Inmultiri de matrici

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

$$G = N^3$$

Matrix multiply

Atenție avem N^2 elemente

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

$$G = N^3$$

Matrix multiply

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Complexitate?

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Complexitate?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$S = \frac{N^3}{\frac{N^3}{P}}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$S = P$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Super-linear speedup?

$S > P?$

- *"... the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude."*

Amdhal, 1967

- *"... speedup should be measured by scaling the problem to the number of processors, not by fixing the problem size."*

Gustafson, 1988